Project Thesis submitted in support of Bachelor of Engineering in Electronics Engineering with Communications

Neural Network Based Adaptive Equalisation Techniques

Final Year Project Thesis

by - IIIE

Project Supervisor -

28 April, 1995

SUMMARY

This project investigates the application of Artificial Neural Networks to the field of adaptive equalisation. The report begins by explaining the causes of Intersymbol Interference and how equalisers are used to counter act it. Next, a procedure for testing and comparing equalisers is described. The project then looks at the performance of four types of neural network as equalisers, namely ADALINE, backpropagation, recurrent an radial-basis function networks.

The results of a fair comparison of the networks under real life conditions are presented at the end of the report and from these it is concluded that artificial neural networks are unsuitable of channel equalisation.

CONTENTS

1. INTRODUCTION 1
1.1 INTERSYMBOL INTERFERENCE 1
1.2 CHANNEL EQUALISATION 1
1.3 METHODS OF ADAPTIVE EQUALISATION 1
1.4 PROJECT AIMS 2
1.5 SIMULATION TOOLS 2

2. MODELLING AN ISI CHANNEL 3
2.1 BASIC CHANNEL REPRESENTATION 3
2.2 CHANNEL GAUSSIAN NOISE SOURCE 3
2.3 ISI MODEL 4

3. ISI CHANNEL EQUALISATION 6
3.1 EQUALISING ISI 6
3.2 EQUALISER TEST PROCEDURE 7
3.3 GENERATION OF THE TRAINING SEQUENCE 7
3.4 EQUALISER TESTING AND COMPARISON 8

4. FEEDFORWARD ARTIFICIAL NEURAL NETWORKS 9
4.1 GENERAL FEEDFORWARD NEURAL NETWORK STRUCTURE 9
4.2 PERCEPTRON MODEL 9
4.3 TRAINING FEEDFORWARD ANNS 11
4.4 USING FEEDFORWARD ANNS IN EQUALISATION 12

5. ADALINE EQUALISER 15
5.1 THE ADAPTIVE LINEAR ELEMENT 15
5.2 THE ADALINE AND LMS IN EQUALISATION 15
5.3 TESTING THE ADALINE EQUALISER 16
5.4 PROBLEMS WITH A LINEAR EQUALISER 17

6. BACKPROPAGATION EQUALISER 20
6.1 THE BACKPROPAGATION ALGORITHM 20
6.2 TESTING THE BACKPROPAGATION EQUALISER 21
6.3 BATCH MODE TRAINING OF MLPS 22

7. RECURRENT NEURAL NETWORK EQUALISER 24
7.1 RECURRENT NETWORK STRUCTURE 24
7.2 RNN TRAINING ALGORITHM 24
7.3 TESTING THE RNN EQUALISER 25

8. RADIAL-BASIS FUNCTION EQUALISER 26
8.1 RADIAL-BASIS FUNCTION NODE 26
8.2 RBF NETWORKS IN EQUALISATION 26
8.3 RBF EQUALISER SIMULATION 27

9. ASSESSMENT OF ANN EQUALISER TECHNIQUES 29

10. CONCLUSIONS 31

REFERENCES 32

APPENDIX A - LMS DERIVATION 33

APPENDIX B - PROGRAM LISTINGS 35

NOTATION

n	Bit number of transmitted signal.
$d(n)$	Preamble signal.
$c(n)$	Output of channel model transfer function.
$g(n)$	Gaussian noise signal.
μ_m	Mean of a random signal.
σ^2	Variance of a random signal.
$x(n)$	Received signal.
$y(n)$	Analogue output of equaliser.
$o(n)$	Digital output of equaliser.
$H[z]$	Channel model transfer function
M	Number of taps in channel model FIR filter.
w_i	Constant weight i in a FIR filter.
$w_{ij}(n)$	Weight between node i and node j in a MLP ANN.
η	Learning rate for training algorithms.
I	Number of inputs to equaliser.
$y_j(n)$	Output of node j in an MLP.
$v_j(n)$	Sum of weighted inputs to node j in an MLP.
$\varphi[\cdot]$	Activation function.
$\mathcal{E}(n)$	Sum of squared errors of ANN.
$\delta_j(n)$	Local derivative of weights w.r.t $\mathcal{E}(n)$ at node j in a MLP.
$\pi_{ij}^k(n)$	Sensitivity factor for output k w.r.t. weight $w_{ij}(n)$ in a RNN.
δ_{ij}	Kronecker delta.
$\|\cdot\|$	Euclidean distance function.
σ_w	Width of Gaussian function in RBF network.

1. Introduction

1.1 Intersymbol interference

When transmitting a digital signal through a channel, the received signal will be corrupted and distorted due to the different characteristics of that channel. Channel corruption's include thermal noise, impulse noise and fading. Signals can be distorted by transformations such as frequency translation, non-linear or harmonic distortion and time dispersion[1]. The need for greater information capacity has led to techniques in counteracting these distortions and corruptions to attain lower bit error rates for higher speed transmissions.

This project is concerned with comparing different methods of cancelling InterSymbol Interference (ISI). ISI results from the time dispersion of a channel such as a telephone line, or from multipath propagation in a channel such as a High Frequency radio transmission[2]. In either case a transmitted symbol will, by the time it reaches the receiver, extend beyond the time interval used to represent that symbol. Figure 1.1 shows an example of the effect of time distortion on a symbol transmitted down a telephone line. It can be seen that the overlap into the neighbouring symbol intervals create will errors and is undesirable.

1.2 Channel equalisation

A method called equalisation is used to remove ISI from a received signal. A filter is added to the front end of the receiver that attempts to regenerate the originally transmitted data stream. The main problem with equalisation is that the dispersion characteristics of the channel are rarely known in advance, and may change with time. Therefore the equaliser must "learn" the channel's characteristics and compensate for them, i.e. it should be adaptive.

1.3 Methods of adaptive equalisation

Since the mid 1960's a large effort has been made in the research of different methods of adaptive equalisation. The earliest form of equaliser is the transversal equaliser[3], based on a Finite Impulse Response (FIR) filter, and uses training algorithms such as Least Mean Squares

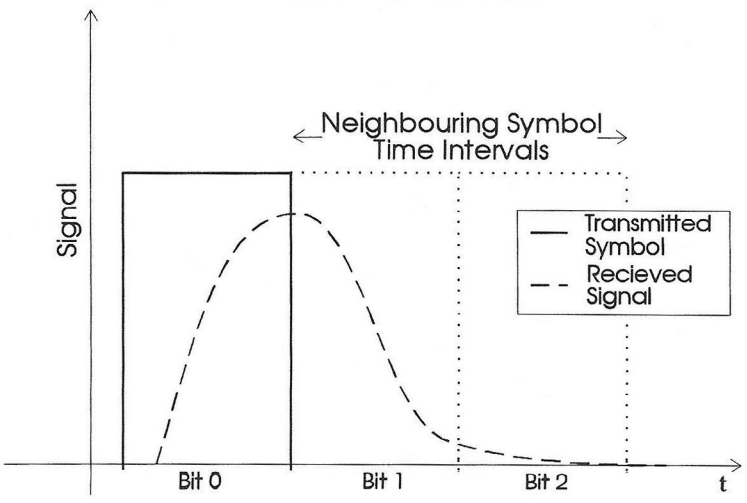

Figure 1.1 - Effect of ISI on a transmitted symbol.

(LMS)[4] and Recursive Least Squares (RLS). These linear equalisers, however have limitations and so non-linear techniques such as the Decision Feedback Equaliser (DFE)[5] were developed in the late 1960's. The mid 1980's saw a resurfaced interest in Artificial Neural Networks (ANNs)[a], and, amongst many other applications, these non-linear structures were applied to the field of adaptive equalisation[6]. Research has since investigated any benefits that different types of ANN may give over the more traditional forms of equaliser.

1.4 Project aims

The aim of this project is to perform a comparison of several types of ANN used as adaptive equalisers The project, over 30 weeks, was planned to follow the course shown below:

- Investigate the modelling of a channel that introduces ISI to a signal.
- Investigate various types of ANN.
- Apply the ANNs that appear appropriate to equalisation.
- Compare the performance of all equalisers in under realistic channel conditions and conclude any benefits that each type of ANN may posses.

1.5 Simulation tools

MATLAB version 4.0 is a very powerful, high level, mathematics language and allows efficient presentation of results. Although MATLAB proved too slow for the actual simulation programs (simulations running into tens of hours), all investigations were initially written using MATLAB and its neural network toolbox[7].

To achieve a shorter simulation time, the MATLAB programs were translated to C. These programs were first written as Dynamic Link Libraries (.DLL files) that linked to MATLAB. However, these DLL's were found to be unstable in the windows operating system and took too long to debug. The simulations were finally compiled as stand alone DOS programs using Borland C/C++ v3.1. Data is transferred to and from these programs by means of a text files. All program listings are presented in appendix B.

[a] Research into ANNs was largely put aside after Minsky and Papert published their book "Perceptrons" in 1968 showing the limitations of the ANN structures known at that time. These limitations were overcome with the introduction of Multi-Layer Perceptron (MLP) networks.

2. Modelling an ISI Channel

This section describes how a digital communications channel that suffers from ISI is modelled in a simple and easily coded way.

2.1 Basic channel representation

Figure 2.1 shows a block diagram of a simple channel model represented by the channel transfer function, $h[\cdot]$ and the additive Gaussian noise, $g(t)$. The data transmitted, $d(t)$, is passed through the channel and is received as $x(t)$.

$$x(t) = h[d(t)] + g(t) \tag{2-1}$$

Although modern transmission systems use techniques such as Frequency Shift Keying (FSK) and Quadrature Phase Shift Keying (QPSK), it is easiest to model ISI using baseband Pulse Amplitude Modulation (PAM). Samples of the analogue signals are taken every T seconds, where T is the symbol period of $d(t)$. Figure 2.2 shows the received signal $x(t)$, used in figure 1.1, sampled in the middle of each symbol interval to give a discrete signal $x(nT)$, where n is the symbol number. Therefore the channel model, $h[\cdot]$ becomes a discrete transform, $H[z]$. The discrete version of equation (2-1) is

$$x(nT) = d(nT)H[z] + g(nT) \tag{2-2}$$

Because T is a constant, $d(nT)$, $g(nT)$ and $x(nT)$ can be written as $d(n)$, $g(n)$ and $x(n)$ respectively, where n represents the time that symbol number n is transmitted. Equation (2-2) is re-written as

$$x(n) = d(n)H[z] + g(n) \tag{2-3}$$

A simple model for a channel can now be simulated by finding a way to code the transform $H[z]$ and the noise source $g(n)$.

2.2 Channel Gaussian noise source

The Gaussian noise signal $g(n)$ has a normal distribution and zero mean[8]. The Signal to Noise Ratio (SNR) of the channel is measured in decibels and is given in reference [9] by:

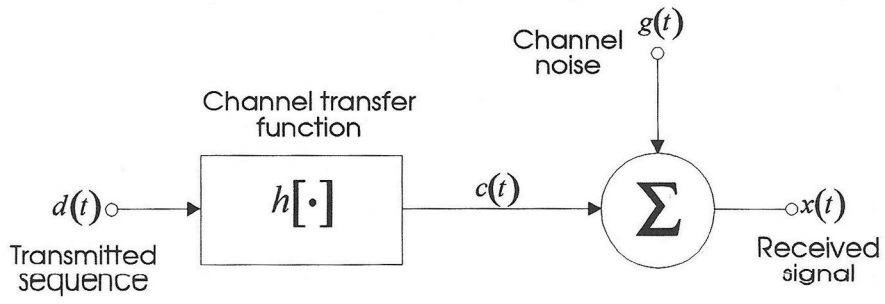

Figure 2.1 - Simple model of a transmission channel.

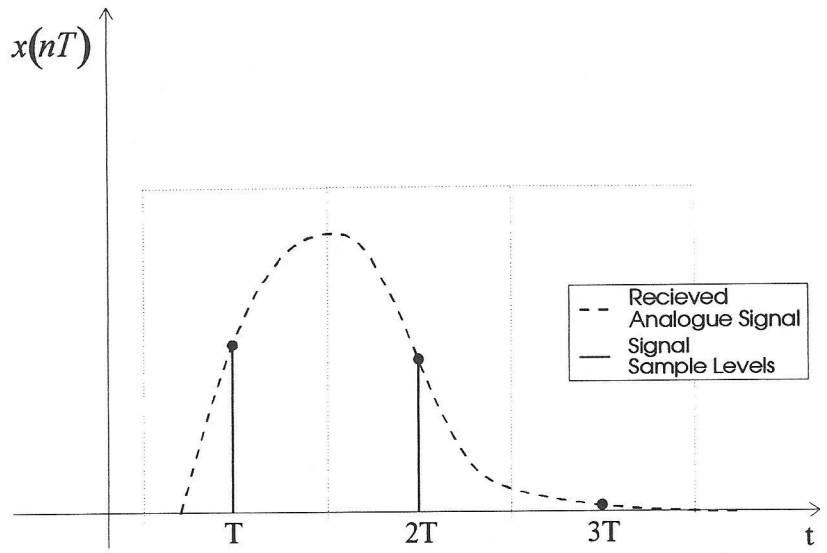

Figure 2.2 - Discrete samples of a received signal

$$SNR\ (\text{dB}) = 10\log_{10}\left(\frac{1}{\sigma^2}\right)$$

where σ^2 is the random signal variance. By setting the SNR in dBs, the variance can be calculated by

$$\sigma^2 = 10^{\frac{-SNR}{10}}$$

C does not provide a function for generating normally distributed random numbers. However reference [9] gives a method of calculating a random value with normal probability from two numbers generated from a uniformly distributed random number generator,

$$g = \mu_m + \cos(2\pi u_1)\sqrt{(-2\sigma^2 \ln u_2)} \qquad (2\text{-}4)$$

where g is the random number with normal probability, u_1 and u_2 are the two numbers with uniform probability, and μ_m is the mean. The random number generator on an IBM-PC can be considered uniform, and u_1 and u_2 are generated by the C function `rand()`. Figure 2.3 shows the probability distribution of a sample of 8192 random numbers generated by equation (2-4), with a mean of zero and a variance of 0.01. It shows an acceptable normal distribution when compared to the ideal Gaussian distribution plotted on the same axis.

2.3 ISI model

The channel transform function, $H[z]$, is often represented as a series of delays:

$$H[z] = 0.3482 + 0.8704z^{-1} + 0.3482z^{-1}$$

This makes the FIR filter ideal for modelling $H[z]$. Figure 2.4 shows an M tap FIR filter where the delays are equal to the symbol period of the transmitted signal. The weights on the filter are set to the coefficients of the $H[z]$ expression:

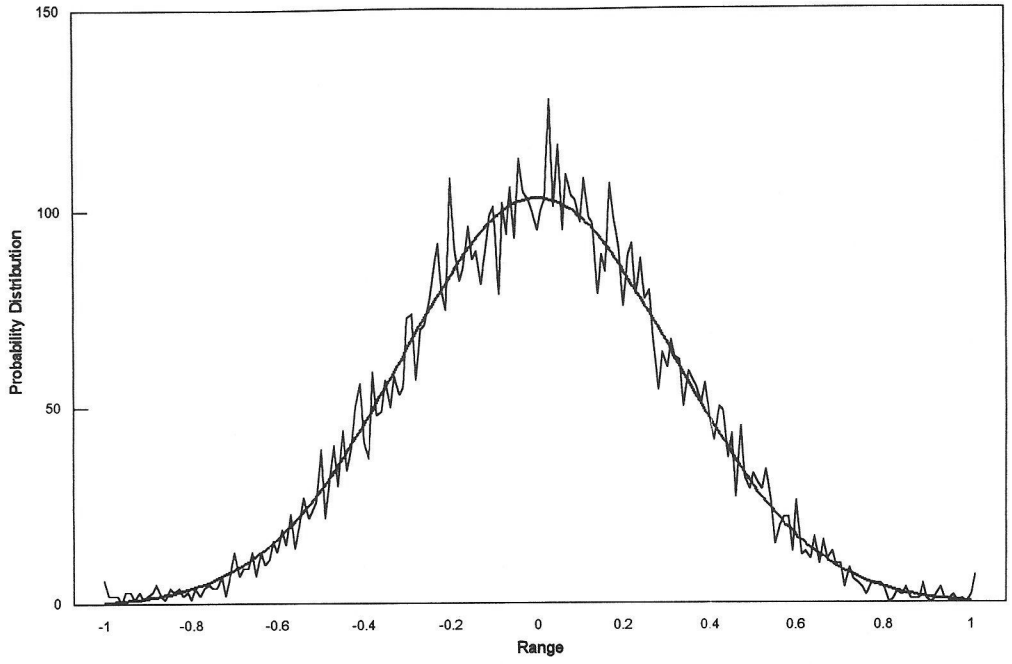

Figure 2.3 - Experimental Gaussian noise plotted against ideal curve.

$$H[z] = w_0 + w_1 z^{-1} + w_2 z^{-2} + \ldots + w_{M-1} z^{-(M-1)}$$

The output of the FIR filter, $c(n)$, is given by the sum of the values on the delay line multiplied by their respective weights.

$$c(n) = \sum_{l=0}^{M-1} w_l d(n-l)$$

Although $H[z]$ may have 10 or more coefficients an a real channel, equaliser testing is performed on 2 or 3 coefficient models to keep down simulation times and keep the training data simple.

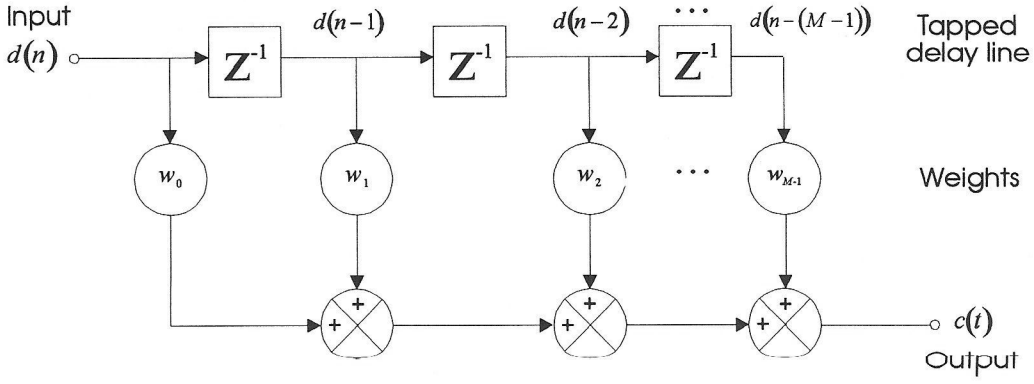

Figure 2.4 - M tap FIR filter used to model channel's transfer response.

3. ISI Channel Equalisation

In order to make a comparison of different equalisation techniques, a standard procedure must be used to test them. This section describes how an equaliser fits into a communication system and uses this to define a test procedure for this project.

3.1 Equalising ISI

Figure 3.1 shows a communications system including the channel model with transmitted signal $d(n)$, received signal $x(n)$ and equaliser output $y(n)$. During training a known sequence is transmitted and the error signal, $e(n)$, is calculated as the difference between the desired output of the equaliser, $d(n)$, and the actual output of the equaliser, $y(n)$. The aim of adaptive equalisation is to minimise the error by the end of the training period. Obviously, the less time spent on training the better, and the convergence rate of the error to a minimum is one of the main points of comparison between equalisers.

Once the equaliser has been trained, information data can be transmitted. During this period, the two symbol states have an equal probability of being transmitted:

$$P[d(n)=+1]=P[d(n)=-1]=0.5 \tag{3-1}$$

The output of the equaliser is fed to a hard-limiter that cleans up $y(n)$ into +1's and -1's. The output of this decision device is labelled $o(n)$:

$$o(n)=\begin{cases}+1 & \text{for } y(n)\geq 0\\ -1 & \text{for } y(n)<0\end{cases} \tag{3-2}$$

If no noise is present and the equaliser has been trained correctly then no errors will occur. However, in noisy environments a trained equaliser can be forced into the wrong decision by large noise spikes. The Bit Error Rate (BER) is tested after training by transmitting a random signal of 10,000 bits that satisfies equation (3-1). The number of symbols that the equaliser incorrectly classifies is counted and divided by 10,000 to calculate the BER.

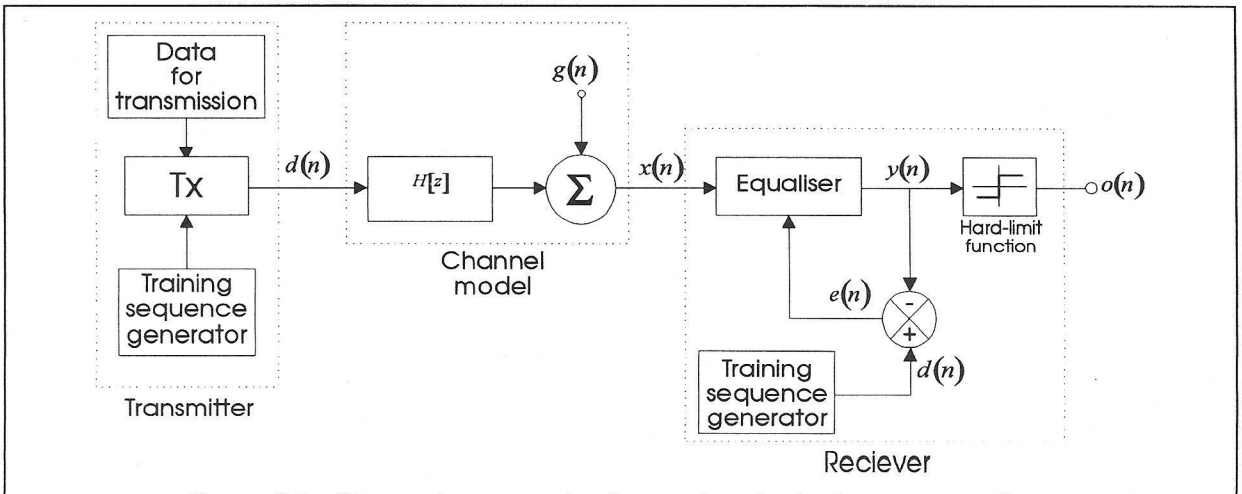

Figure 3.1 - General communication system including an equaliser.

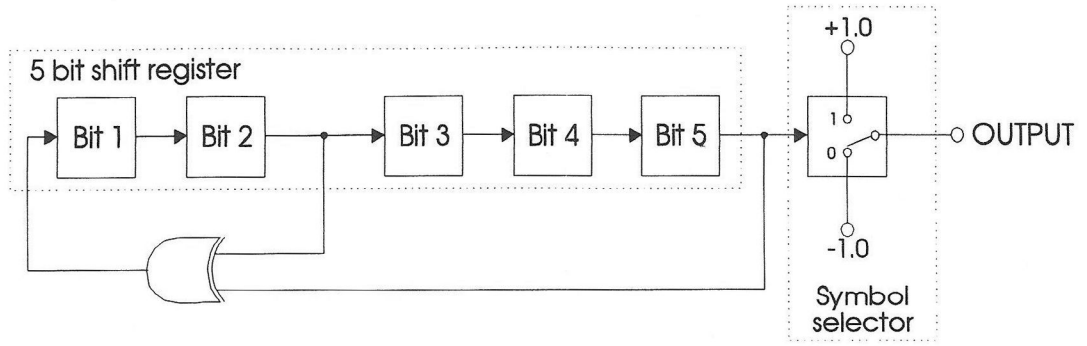

Figure 3.2 - 5 bit pseudo-random sequence generator.

3.2 Equaliser test procedure[10]

The equalisers investigated in this project are all tested in the same way and compared under similar conditions. Initially a training sequence of 1,000 bits is transmitted and the square of the error signal, $e(n)$ is recorded. Following training, the 10,000 test sequence bits discussed in section 3.1 are transmitted and the BER is calculated.

This two part test is performed 100 times. The square error signal is averaged to remove the noise components and results in the ensemble squared error signal. The mean of the BER is also recorded.

Occasionally an ANN will not train and this will effect the mean results. Included in the simulations is a test to check if the ANN completes training successfully. If this test is failed, then the ANN is reset and trained again.

3.3 Generation of the training sequence

Both states of the bits generated in training data should have an equal probability of occurring. The sequence should also be known by the equaliser. A convenient solution to these restrictions on the training sequence is the Pseudo-random Noise (PN) generator. The generator is based on a shift register, an example is shown in figure 3.2, based on a 5-bit shift register. Bit numbers 2 and 5 are XORed and fed back to bit 0.

If the shift register is primed with all 1's then a repetitive sequence of 1's and 0's is produced. Because of the logical structure of the generator the sequence will always be the same. Therefore, one PN generator can be used to generate a training sequence at the transmitter, and another at the receiver can be synchronised to produce the desired signal.

The 5 bit generator's sequence has a period of 31 bits. This length should give enough training patterns within the whole preamble length without having too many sequential 1's or 0's. Before the PN sequence is transmitted, all 0's are converted to -1's to give the sequence an approximate mean about zero.

3.4 Equaliser testing and comparison

Whilst a type of equaliser is being investigated it is trained using two channels. Channel one is:

$$H[z] = 1.0 + 0.5z^{-1}$$

and channel 2 is:

$$H[z] = 0.5 + 1.0z^{-1}$$

All the equalisers are tested on a real life channel when compared with each. The coefficients of this model are listed in table 3.1. Section 9 deals with how the equalisers compare with this channel model.

Table 3.1 - Coefficients of real life channel model.

Order of delay	Co-efficient	Order of delay	Co-efficient
z^0	0.047309	z^{-5}	0.100099
z^{-1}	0.320619	z^{-6}	0.040872
z^{-2}	0.782586	z^{-7}	0.037772
z^{-3}	0.426974	z^{-8}	0.020498
z^{-4}	0.293453	z^{-9}	0.010807

4. Feedforward Artificial Neural Networks[11][12][13][14]

This section introduces the feedforward ANN structure and explains the notation used in this report.

4.1 General feedforward neural network structure

A feedforward ANN consists of several layers of nodes. These nodes, or perceptrons, are modelled on the neurons that are found in a living brain. There are three types of nodes:

1. **Input nodes** act as buffers, i.e. they have one output that follows the one input exactly. Input nodes, as their name suggests, form the first layer of the ANN.

2. **Output nodes** make up the final layer of the ANN and the outputs of these nodes are the network's outputs. These nodes have the structure of the perceptron model.

3. **Hidden nodes** are so called because neither their inputs nor their output are seen by the "outside world". Other than this they are physically the same as output nodes in that they also follow the perceptron model.

The each output and hidden node is connected to the outputs of all the nodes in the previous layer as shown in figure 4.1. This creates a large set of interconnections, and it is this that makes the neural network a complex, but powerful parallel structure.

4.2 Perceptron model

A simple diagram of a biological neuron can be seen in figure 4.2. It has many inputs and one output which fires when the sum of the inputs exceed a threshold.

The general structure of perceptron model can be seen in figure 4.3. It is indexed as node j from a layer of similar nodes. There are I inputs, $x_0(n)$ to $x_{I-1}(n)$ that are multiplied by their own

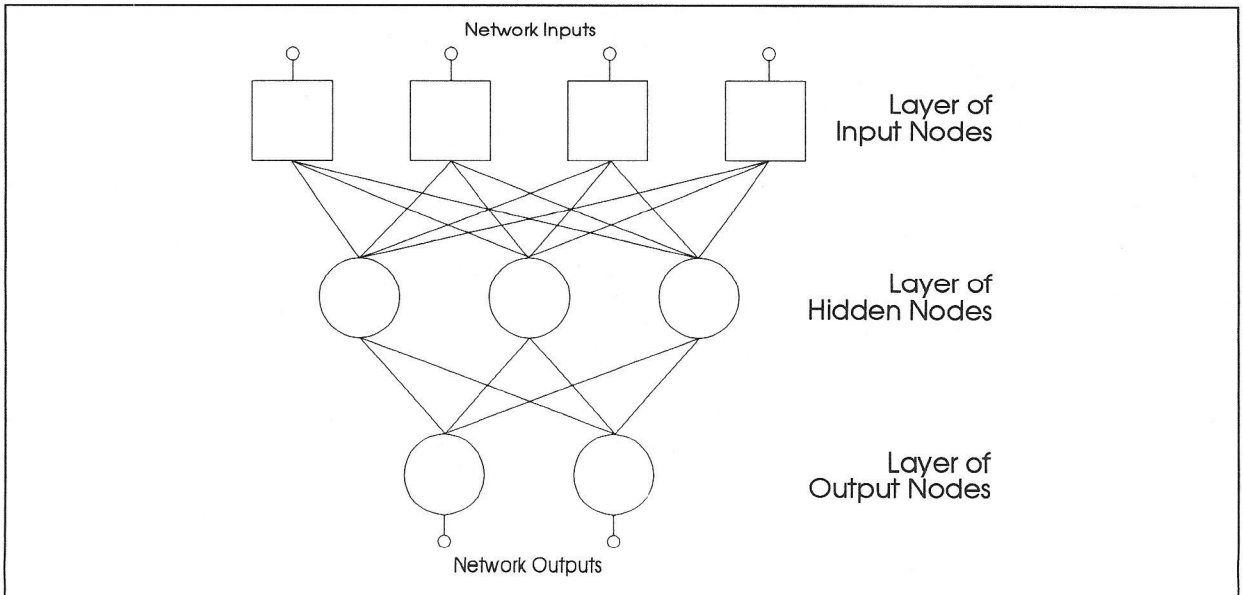

Figure 4.1 - General layout of a feedforward ANN.

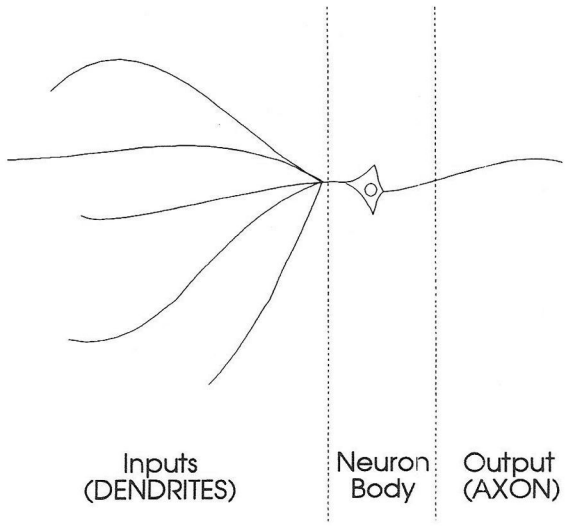

Figure 4.2 - Simple diagram of a biological neuron.

respective weights, $w_{0j}(n)$ to $w_{(I-1)j}(n)$. These products are summed by the summing unit and this total, $v_j(n)$, is passed through the activation function, $\varphi_j[\cdot]$.The output of the node, $y_j(n)$, only "fires" if the input to $\varphi_j[\cdot]$ exceeds a certain threshold value, θ_j. This action is described in equation form as:

$$y_j(n) = \varphi_j\left[v_j(n) - \theta_j\right] \tag{4-1}$$

$$v_j(n) = \sum_{i=0}^{I} w_{ij} x_i \tag{4-2}$$

Figure 4.4 shows how these perceptron nodes are put together to form a Multi-Layer

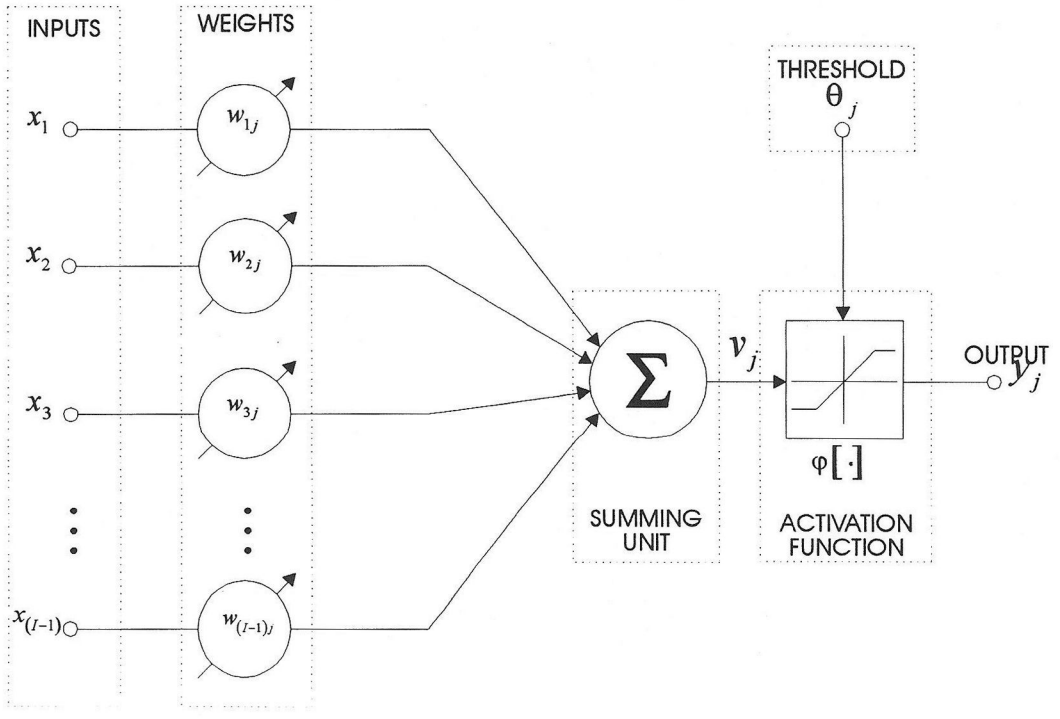

Figure 4.3 - Perceptron model of a neuron

Perceptron (MLP) feedforward network. In this case there is a layer of I input nodes, followed by one hidden layer containing J hidden nodes, then finally the output layer with K output nodes.

The threshold function of the perceptrons is simulated by using an extra node input. This input is connected to a bias node added to the previous layer. The bias node's output is fixed at -1. This makes coding the ANN much simpler. For example, the weight connecting node k to the bias node J in the layer above represents the threshold value for node k, that is:

$$w_{Jk} = \theta_k \tag{4-3}$$

When a set of inputs is applied to the network the signals are fed forward through the network by calculating the output of each node, layer by layer, until the outputs of the ANN are known. Combining equations (4-1), (4-2) and (4-3), the output of node j, $y_j(n)$, can be calculated from the outputs of the nodes in the previous layer ($y_i(n)$, $0 \le i \le I$):

$$y_j(n) = \varphi\left[\sum_{i=0}^{I} y_i(n) w_{ij}(n)\right] \tag{4-4}$$

where I is the number of nodes in the previous layer, including the bias node.

4.3 Training feedforward ANNs

The ANN is initialised by setting the weight values to small uniformly distributed random numbers. Then the ANN is trained by adjusting its weights to reduce the error of the output. There are two methods of training an ANN:

1. Supervised training is used when the desired output of the ANN is known for each set of inputs. The difference between the desired output and the actual ANN output can be used to update the weights in the network to reduce this error.

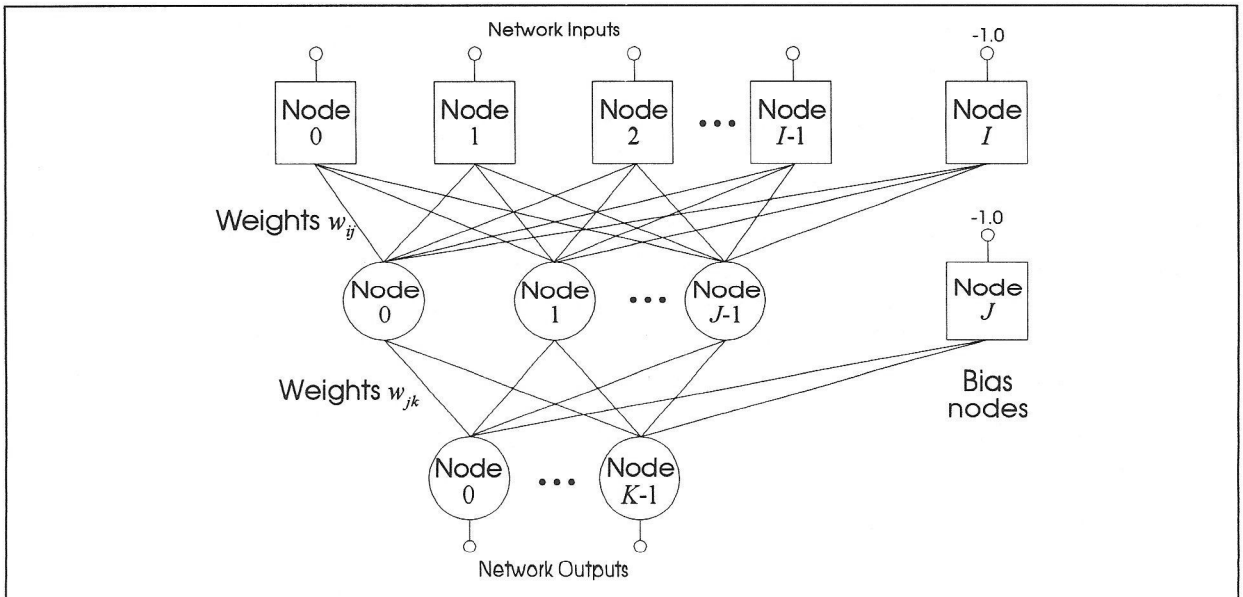

Figure 4.4 - General layout of a feedforward MLP network with bias nodes.

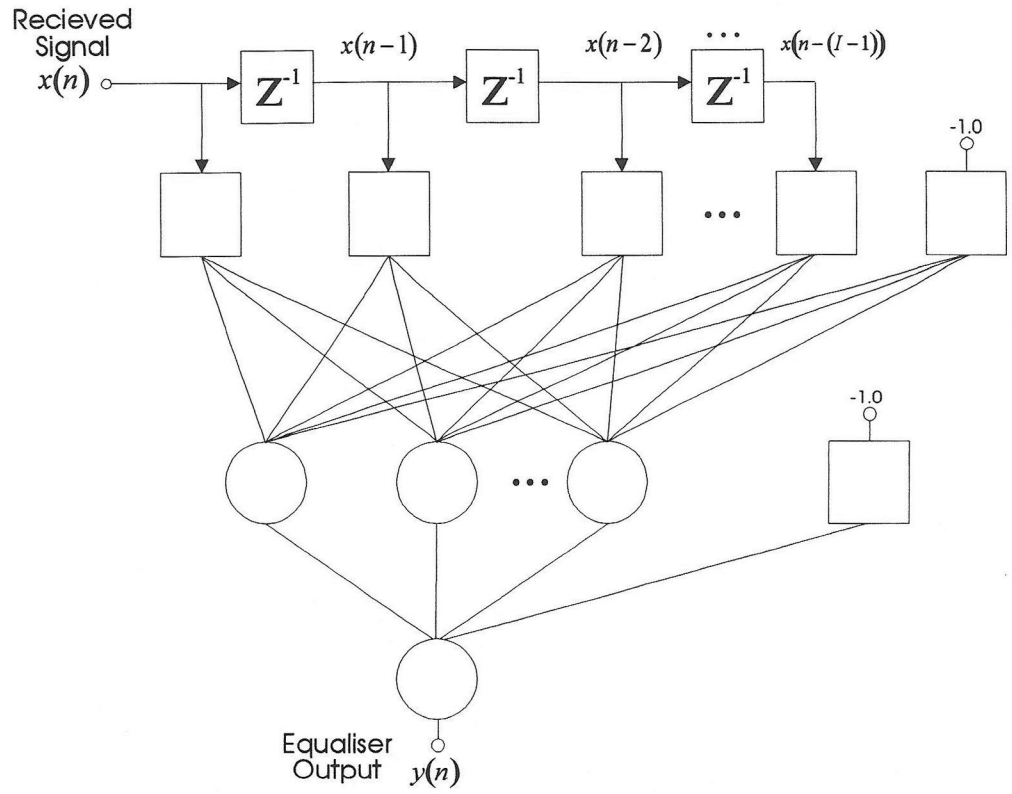

Figure 4.5 - General layout of feedforward network used as an adaptive equaliser.

2. Unsupervised training is used when the desired outputs of the ANN are unknown and the ANN tries to classify the input patterns into different groups by itself. Unsupervised learning is not used for equalisation in this project.

4.4 Using feedforward ANNs in equalisation

Figure 4.5 shows an example of a feedforward ANN used as an adaptive equaliser. There are I inputs connected to a tapped delay line with delays of one symbol period. The ANN has one hidden layer and an output layer with one output node. At time n, the received signal $x(n)$ is passed into the delay line. The signals on the taps are fed through the network to produce the equaliser output of $y(n)$.

The inputs to an ANN are classified because the ANN approximates some kind of decision function. During channel equalisation the network is required to classify the pattern made by the signal on the delay into one of two classes:

1. Those input patterns result from +1 being transmitted at time n.

2. Those input patterns result from -1 being transmitted at time n.

If an ANN has two inputs, and is trained on channel 1 (see section 3.4) with no noise, then there are 8 possible patterns that can be presented to the ANN. They are listed in table 4.1 with the sequence of transmitted symbols that generated them. The input patterns are plotted in blue in 2 dimensional space in figure 4.6. A $\times$ implies that co-ordinate should be classified as -1, and

a ● should be classified as +1. Also shown on the plot are 1000 samples of received data with a SNR of 16 dB, giving an idea of the spread of received co-ordinates under noisy conditions.

Bayes presented a method for constructing an optimum decision function. The decision boundary for the above channel is plotted on figure 4.6 in red. Anything to the left of this boundary is classified by the Bayes function as a -1, everything to the right as a +1. ANNs are function approximaters and figure 4.7 shows the output surface of the Bayes function for the range of co-ordinates in figure 4.6. It is this surface that the ANN should approximate.

Table 4.1 - Possible received patterns and sequences that generated them.

$d(n)$	$d(n-1)$	$d(n-2)$	$x(n)$	$x(n-1)$
-1	-1	-1	-1.5	-1.5
-1	-1	+1	-1.5	-0.5
-1	+1	-1	-0.5	+0.5
-1	+1	+1	-0.5	+1.5
+1	-1	-1	+0.5	-1.5
+1	-1	+1	+0.5	-0.5
+1	+1	-1	+1.5	+0.5
+1	+1	+1	+1.5	+1.5

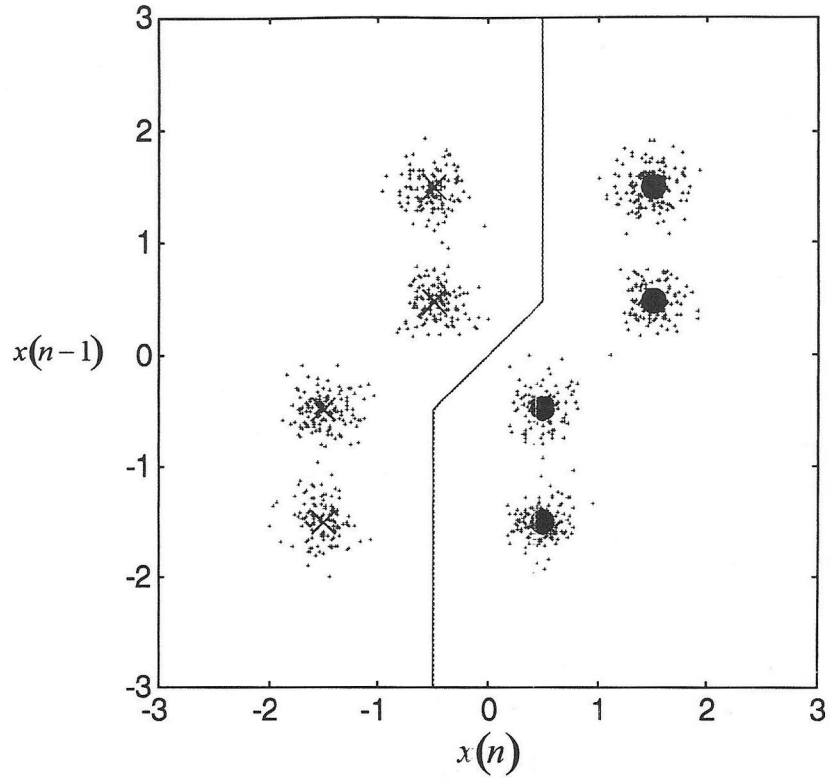

Figure 4.6 - Optimum Bayesian decision boundary for channel 1, plotted with 1000 sample data points.

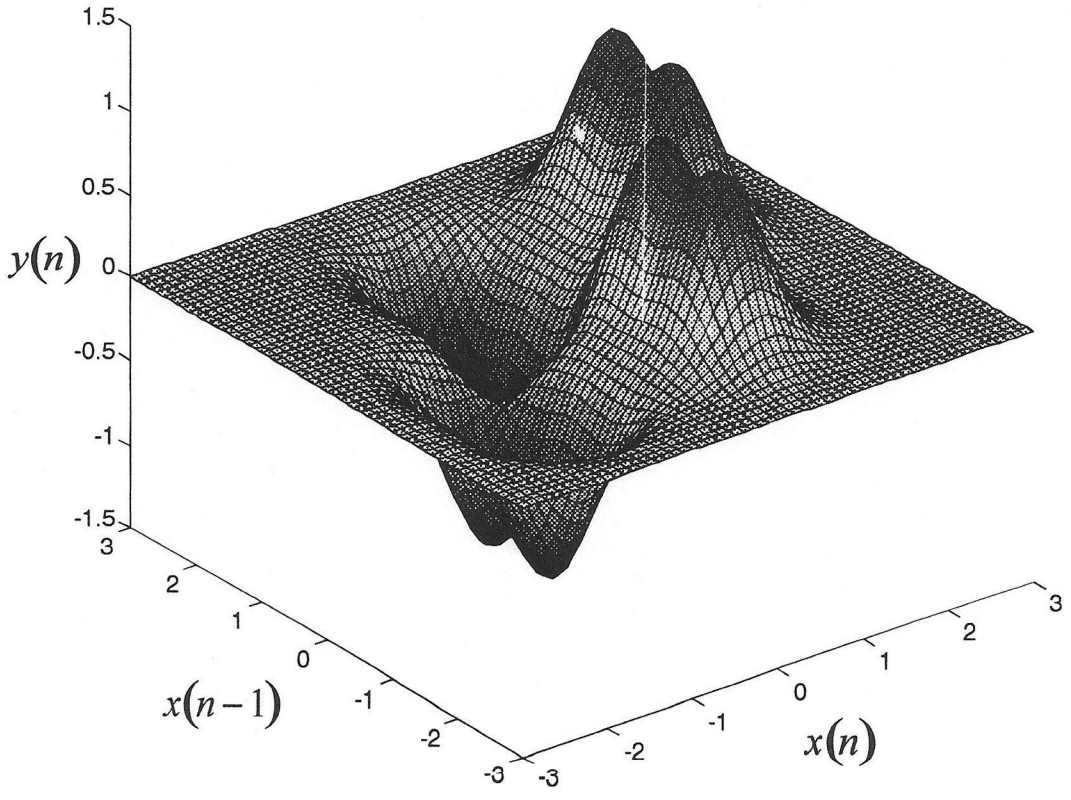

Figure 4.7 - Optimum Bayesian function surface for channel 1.

5. ADALINE equaliser

5.1 The adaptive linear element

The ADAptive LINear Element (ADALINE) is a structure based on the perceptron model shown in figure 4.1. The activation function of an ADALINE is linear:

$$\varphi[v(n)] = v(n)$$

The ADALINE was put forward in the early 1960's by Bernard Widrow[15], together with the Least Means Squares (LMS) algorithm for training ADALINE networks[16].

5.2 The ADALINE and LMS in equalisation

Figure 5.1 shows an ADALINE used as an adaptive equaliser. The network has I inputs, no hidden layer and one ADALINE node in the output node. The output of the network is subtracted from the desired output signal to give the error signal:

$$e(n) = d(n) - y(n)$$

The LMS algorithm is a gradient descent algorithm and is described in more detail in appendix A. It uses $e(n)$ to calculate the next change in the weights:

$$w_{i0}(n+1) = w_{i0}(n) + 2\eta e(n)x(n-1)$$

where η is the learning rate, a constant that limits the amount a weight is changed by. If η is too large then the network will become unstable and the values of the weights will tend to infinity.

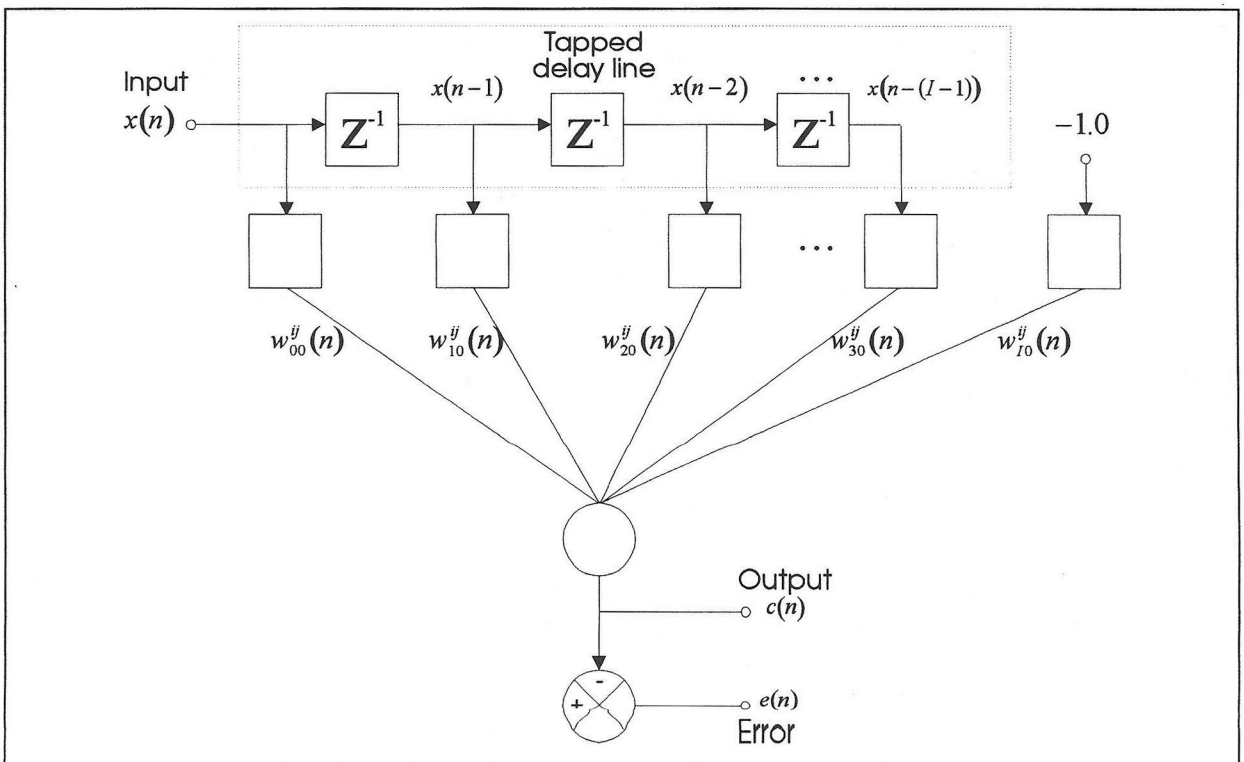

Figure 5.1 - Structure of the ADALINE equaliser.

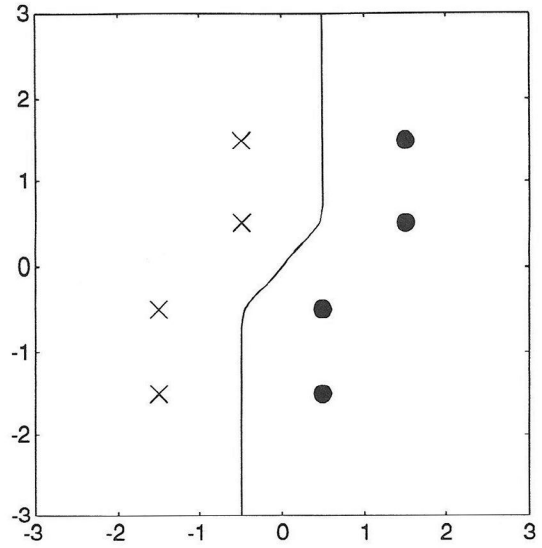

Figure 5.2 - Optimum Bayesian boundary for channel 1.

A small η guarantees convergence, but convergence time takes longer as the value of η decreases.

It is noted that, excluding the bias node, the ADALINE equaliser is identical to the FIR or transversal filter, which is a well proved and established design of adaptive equaliser. Therefore the performance of the ADALINE equaliser is taken as standard and the other equalisers are compared to it.

5.3 Testing the ADALINE equaliser

The ADALINE equaliser was first tested using channel 1, for which, the optimal Bayesian decision boundary for a two input equaliser is shown in figure 5.2. Again a $\times$ represents a co-ordinate that should be classified as -1, and a $\bullet$ as $+1$. The adjustable parameters for the ADALINE equaliser were I, the number of network inputs; SNR, the signal to noise ratio of the channel; and η, the learning rate of the LMS algorithm.

For a constant SNR of 21dBs and a learn rate of 0.01, the number of network inputs were set to 1, 2, 3, 7 and 11 in each test. The resulting error signals are plotted in figure 5.3. It can be seen that increasing the number of inputs reduces the final error of the equaliser. There is a point at which increasing the number of inputs offers little extra benefit, in this example it is 7 inputs. Increasing the number of inputs above 7 would only add to the complexity of the structure.

The effect of altering the learning rate, η , was investigated by setting the SNR to 21 dBs and the number of inputs to 7. Figure 5.4 shows a plot of the error signals for varying η. It can be seen that a learning rate of 0.1 makes the network unstable, and the error never converges. Also, with a learning rate of 0.0001, the network remains stable, but the converge is very slow. Convergence to an acceptable error level at this rate would take tens of thousands of training bits. The optimum learning rate shown is 0.01.

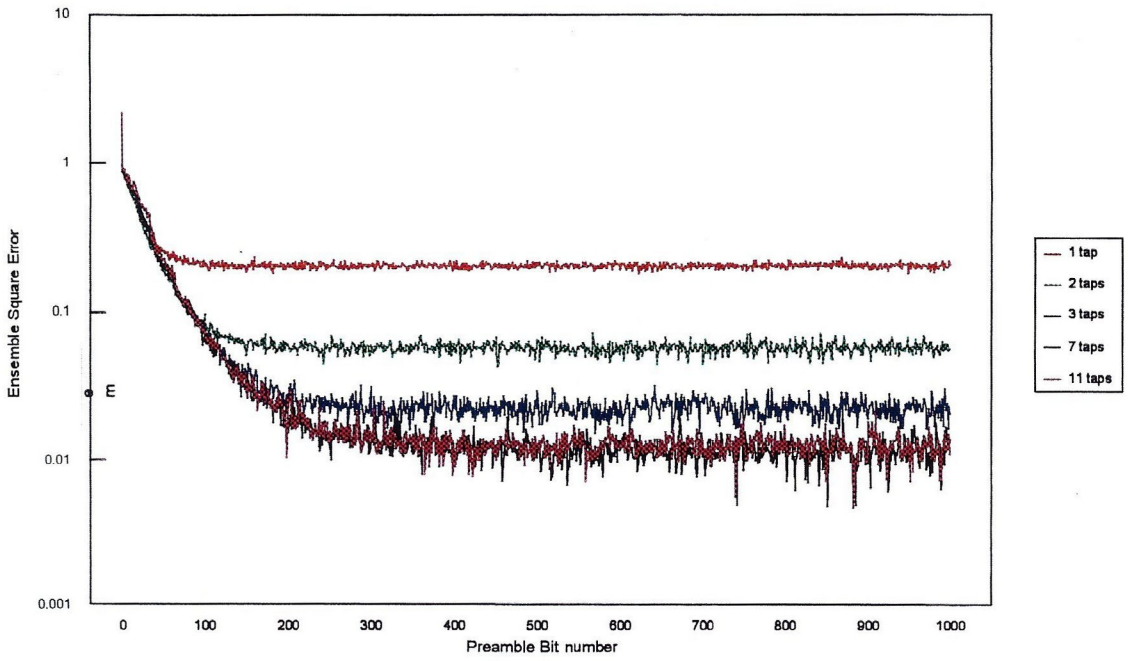

Figure 5.3 - Error signals for ADALINE equaliser when varying the number of inputs.

Using the optimal set up of 7 inputs and a learning rate of 0.01, the ADALINE equaliser was tested over a range of SNRs. The resulting BERs are plotted against the SNRs in figure 5.5, and show a very acceptable performance in noisy conditions.

5.4 Problems with a linear equaliser

The decision boundary shown in figure 5.2 could be re-drawn as a straight line separating the two sets of co-ordinates. This implies that the co-ordinates are linearly separable. The ADALINE equaliser classifies the two sets of co-ordinates with a straight line boundary due to its linear structure. If the ADALINE equaliser is presented with a co-ordinate set shown in figure

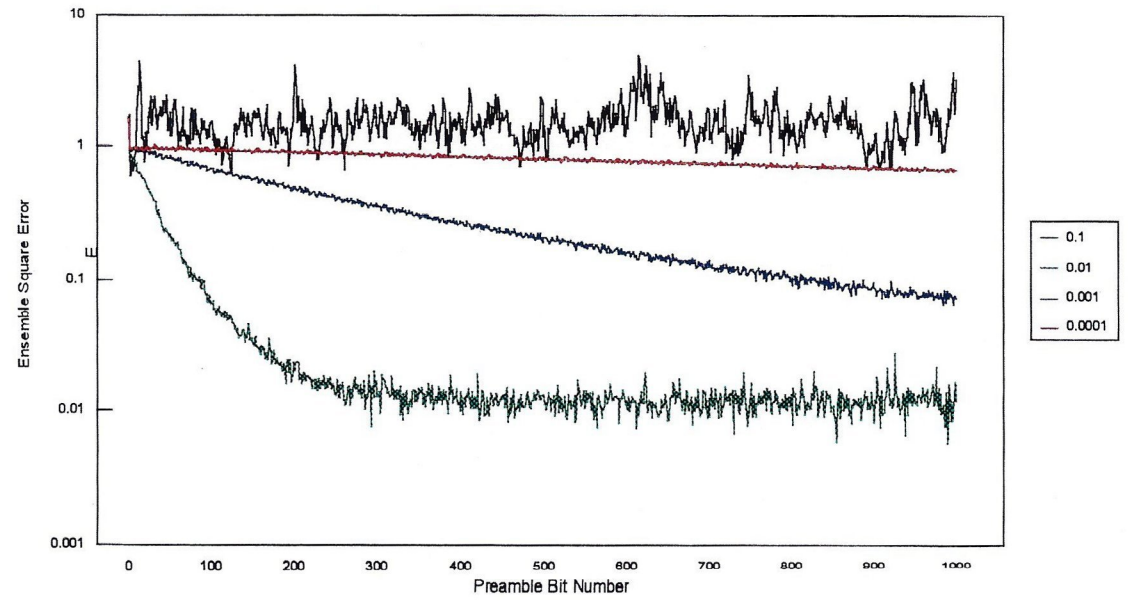

Figure 5.4 - Error signals for ADALINE equaliser when varying the LMS learning rate.

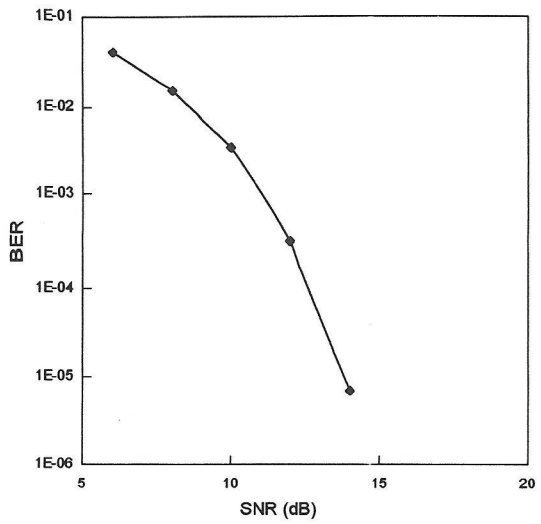

Figure 5.5 - Bit Error Rate plot for a 7 input ADALINE equaliser.

5.6, classification fails because the two co-ordinate sets are not linearly separable.

This problem is however easily solved by viewing the equaliser as a transfer function. In a noiseless channel:

$$x(n) = d(n)H[z]$$

To recover the desired signal, the equaliser must act as the channel inverse $H^{-1}[z]$:

$$\begin{aligned} y(n) &= x(n)H^{-1}[z] \\ &= d(n)H[z]H^{-1}[z] \\ &= d(n) \end{aligned}$$

Reference [17] describes how $H^{-1}[z]$ can be expanded, using Laurent's series, to an infinite series, as follows

$$H^{-1}[z] = a_0 z^b + a_1 z^{b-1} + a_2 z^{b-2} + a_3 z^{b-3} + \ldots$$

where a_n is a constant coefficient and b is a positive or negative integer. If b is greater than zero knowledge of future symbols is required for the inverse function. This makes it non-causal and, of course, unrealisable. This problem is overcome by adding a delay of b symbol periods to a truncated inverse function, it now becomes

$$H^{-1}[z] = z^{-b}\left(a_0 z^b + a_1 z^{b-1} + a_2 z^{b-2} + \ldots + a_{(I-1)} z^{(I-1)}\right)$$

Therefore,

$$\begin{aligned} y(n) &\approx d(n)H[z]H^{-1}[z] \\ &\approx d(n)z^{-b} \end{aligned}$$

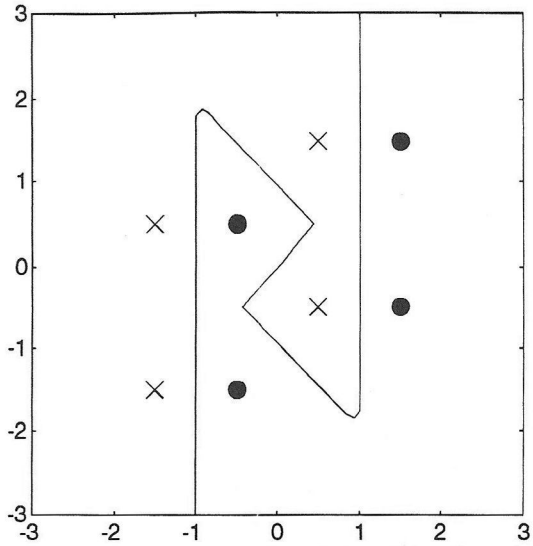

Figure 5.6 - Optimum Bayesian boundary for channel 2.

If the weights in an ADALINE equaliser are used to represent the coefficients, a_0 to $a_{(I-1)}$, of $H^1[z]$, then the network can be trained using the desired response delayed by b symbol periods.

An ADALINE equaliser with 7 inputs and learning rate of 0.01 was tested on the channel 2 with the desired signal delayed by 7 bits. The BER results are shown in figure 5.7 and show that the ADALINE equaliser is equally as effective on this channel as it was with the previous one.

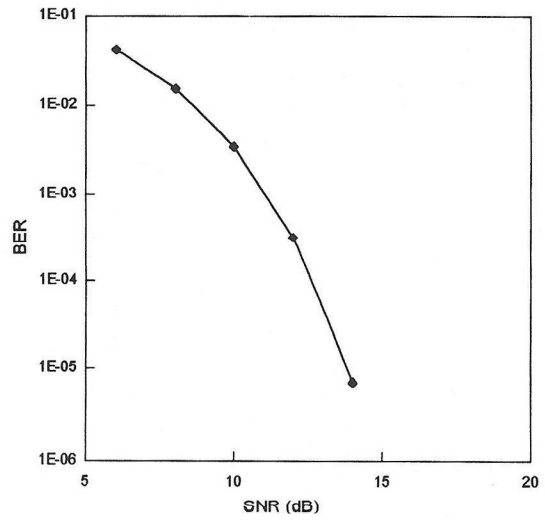

Figure 5.7 - Bit Error Rate plot for a 7 input ADALINE equaliser.

6. Backpropagation Equaliser[14]

The backpropagation algorithm is used on MLP ANNs of any number of layers and has proved to be the most popular algorithm in the field of neural networks. This section aims to provide a brief introduction and the relevance of the backpropagation algorithm to equalisation.

6.1 The Backpropagation Algorithm

If the sum of squared errors of the network at time n is defined as

$$\varepsilon(n) = \frac{1}{2}\sum_{k=0}^{K} e_k^2(n)$$

then the aim of the backpropagation algorithm is to minimise $\varepsilon(n)$ by applying a correction of Δw_{ij} to weight w_{ij} for all weights in the network. It achieves this by estimating the instantaneous gradient of the sum of squared errors with respect to that weight:

$$\frac{\partial \varepsilon(n)}{\partial w_{ij}(n)}$$

which provides the direction of search to a minimum in weight/error space, for the weight w_{ij}. Reference [???] makes use of the chain rule to provide an equation for the local error gradient for node j

$$\delta_j(n) = e_j(n)\varphi'\left[v_j(n)\right]$$

where $e_j(n)$ is the error of the output of node j in the network. The simple gradient descent rule provides the weight correction term,

$$\Delta w_{ij}(n) = \eta \delta_j y_i \qquad (6\text{-}1)$$

where η is the learning rate chosen small enough to avoid instability of the network. Therefore, for the one output node of an MLP used as an equaliser:

$$\Delta w_{i0}(n) = \left(d(n) - y(n)\right)\varphi'\left[v_0(n)\right] \qquad (6\text{-}2)$$

The error signals of each node are propagated back through the hidden nodes in the network by:

$$\delta_i(n) = \varphi'\left[v(n)\right]\sum_{j=0}^{J} \delta_j(n) w_{ij} \qquad (6\text{-}3)$$

where J represents the number of nodes in the second layer and δ_i. is the local derivative for node i. The backpropagation algorithm is as follows:

1. Feed the input signals through the network.
2. Calculate the error of the network output.
3. Calculate the local gradient of the output node using equation (6-2).
4. Calculate the local gradients of the hidden nodes using equation (6-3).
5. Update weights using equation (6-1)
6. Go to step 1 until trained, i.e. when $e(n)$ reaches an acceptable minimum.

The activation function of the MLP is usually taken as the sigmoid function (plotted in figure 6.1),

$$\varphi[v(n)] = \tanh(v(n)) \tag{6-4}$$

because it has a continuous and simple differential,

$$\varphi'[v(n)] = \frac{d}{dv(n)}(\varphi[v(n)]) = 1 - (\varphi[v(n)])^2 \tag{6-5}$$

6.2 Testing the backpropagation equaliser

Reference [6 ???] states that the backpropagation equaliser adapts better than the ADALINE equaliser in noisy conditions. However, when tested the backpropagation equaliser was found to be extremely unreliable. The equaliser failed to converge within 1000 training bits at least once in every five trials, even on noiseless channels. For this reason, temporal training of the backpropagation algorithm was not investigated any further.

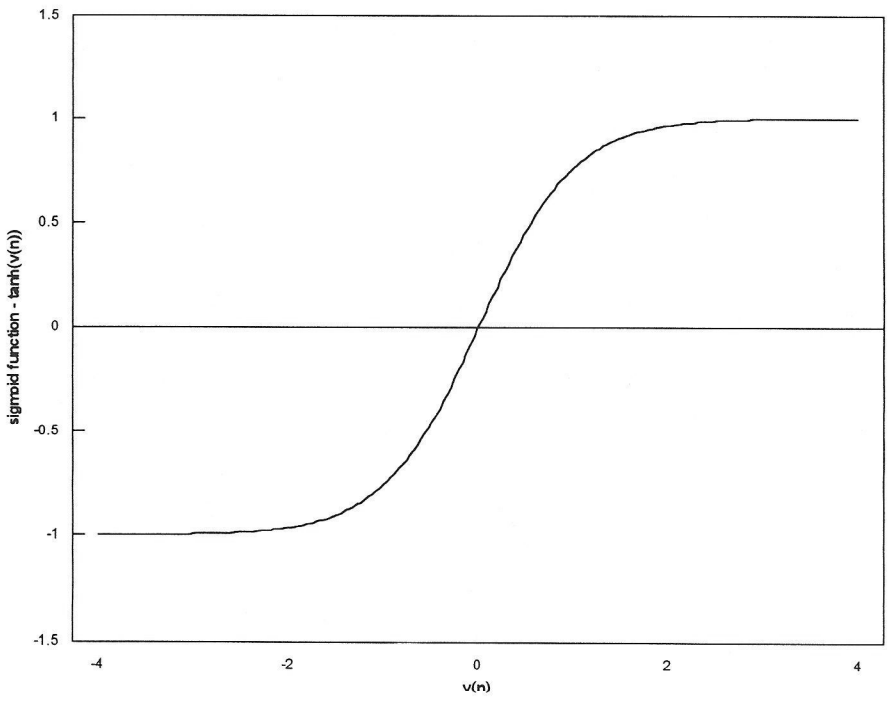

Figure 6.1 - The sigmoid activation function.

6.3 Batch mode training of MLPs

If a set of input patterns is gathered together, the network can be trained repeatedly on this set. One epoch is defined as one presentation of the whole pattern set. If the mean local gradients of the network are computed over an epoch so that the weights are adjusted at the end of that epoch then a much smoother descent of the minimum error is obtained.

This method is a very impractical of realising an equaliser, however it was an interesting investigation to attempt to train an MLP on noisy data such as that presented in figure 4.6. 40 random noisy samples were taken to make up the test set, and the ANN was trained over 1000 epochs. Reference [6] suggests a 2-6-4-1 MLP (2 inputs, 6 hidden nodes in the first hidden layer, 6 in the next, and 1 output).

The initial observation during the simulation was the increased computational complexity of the network over the ADALINE equaliser. Each trial took at least ten times longer than an equivalent ADALINE trial, making it difficult to find available computing time to collect results.

However the backpropagation network converged well in noisy conditions as the ensemble squared error plot in figure 6.2 shows for channel 1. The decision boundary made by the backpropagation equaliser is shown in figure 6.3(a) for channel 1 and figure 6.3(b) for channel 2. The boundary plots includes the optimum Bayesian boundaries for both channels. It can be seen that the boundaries differ from the optimum. In fact, every time the network is trained, the final decision boundary will be different depending on the initial value of the weights. Figure 6.4 shows the MLP's function surface. It is seen to be fairly complex and this is due to the many degrees of freedom within the network.

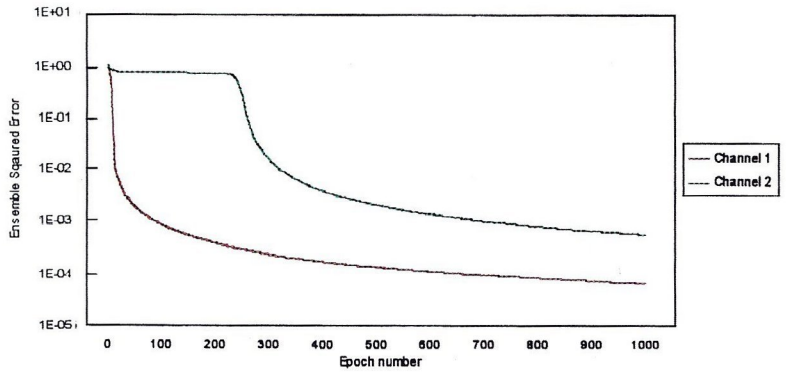

Figure 6.2 - Adaptation curves for backpropagation equaliser

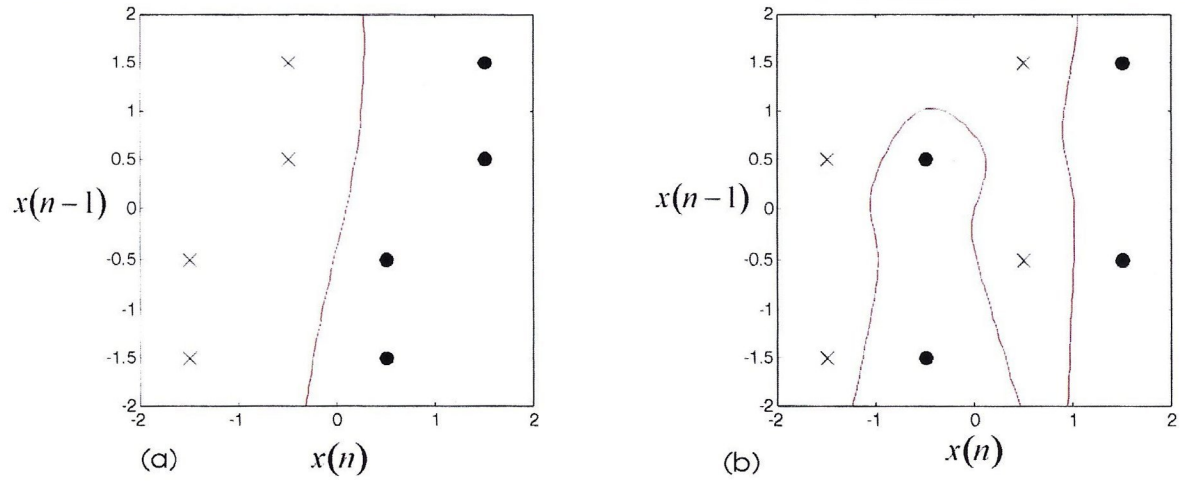

Figure 6.3 - Backpropagation decision boundaries for (a) channel 1, (b) channel 2.

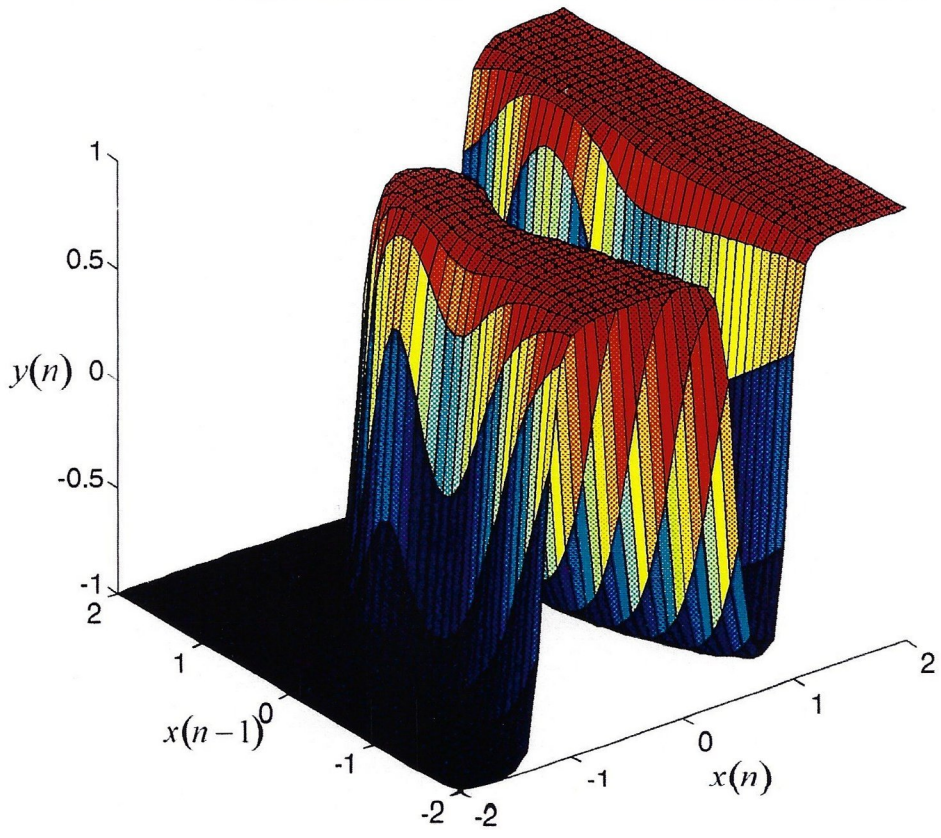

Figure 6.4 - Backpropagation equaliser function surface for channel 2.

7. Recurrent neural network equaliser

7.1 Recurrent network structure

The Recurrent Neural Network structure is based on the standard MLP structure, with the addition of the outputs being fed-back to the inputs via a unit delay. Figure 7.1 shows an example RNN with two inputs. Although there are three output nodes, only one is used as an output to the "outside world". All three output nodes are connected to unit delays and then on to three input nodes to complete the feedback cycle. The input signals are fed through the network as described in section 4.2.

7.2 RNN training algorithm

One algorithm for training RNNs is the Real-Time Recurrent Learning (RTRL) algorithm. Again this is a gradient descent algorithm that uses an estimate of the slope error signal in weight space. Due to the structure of the network, the RTRL algorithm is more complex than backpropagation. For each weight a set of sensitivity values must be computed:

$$\pi_{ij}^{k}(n) = \frac{\partial y_k}{\partial w_{ij}}$$

i.e. the differential of the output of node k, y_k, with respect to weight w_{ij}. References [14][18] derive this as:

$$\pi_{ij}^{k}(n) = \varphi'\left[v_k(n)\right]\left[\sum_{l=0}^{K} w_{ij}(n)\pi_{ij}^{k}(n-1) + \delta_{ik}u_j(n)\right] \qquad (7.1)$$

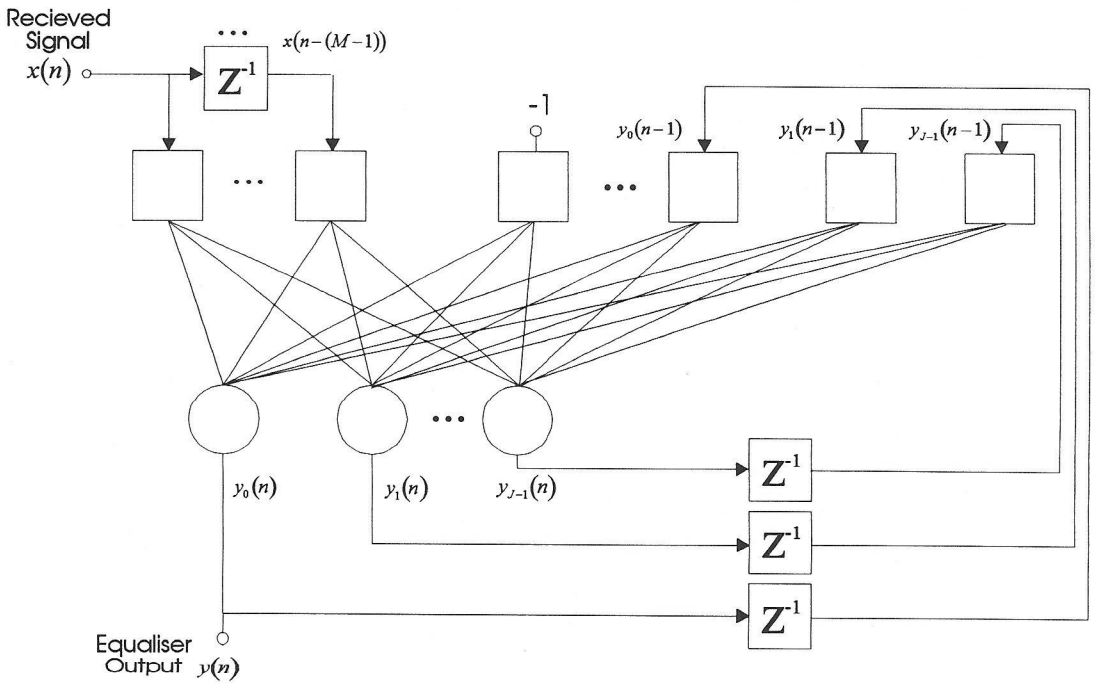

Figure 7.1 - A recurrent neural network used as an adaptive equaliser.

where δ_{ik} is the Kronecker delta which equals 1 if i=k, otherwise equals 0. $u_j(n)$ is the output from input node j. The following initial condition is assumed:

$$\pi_{ij}^{k}(0) = 0$$

The RTRL algorithm proceeds as follows:

1. Feed signals throughout the network.
2. Calculate error signal from the network output:
$$e(n) = d(n) - y(n)$$
3. Calculate set of sensitivities for each weight with equation (7.1).
4. Update the weights with:
$$w_{ij}(n+1) = w_{ij}(n) + \eta e(n)\pi_{ij}^{0}$$
5. Go back to step 1 until trained.

7.3 Testing the RNN equaliser

When the RNN equaliser was tested on both channels 1 and 2 it was found to converge well in noisy situations. However, the RNN was very sensitive to the values for the learning rate and the structure size. If the learning rate was too high then the RNN became unstable very quickly, unlike the backpropagation algorithm which tended to recover after a while.

Figure 7.1 shows the convergence plot for the RNN equaliser for channel 1 with 21 dBs SNR. Figure 7.2 shows the BER performance for channel 1 over a range of SNRs. For both figures the RNN had two inputs, three output nodes and a learning rate of 0.005.

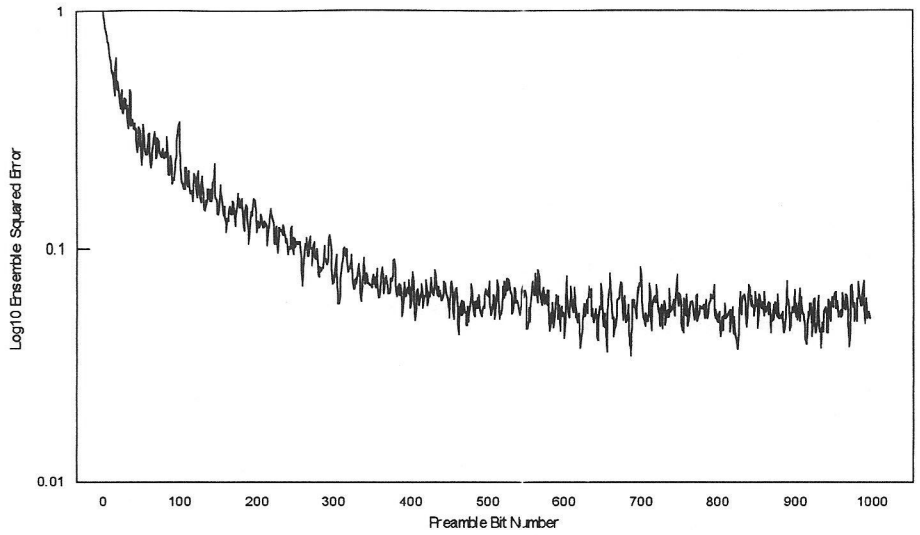

Figure 7.2 - Convergence performance of RNN equaliser on Channel 1.

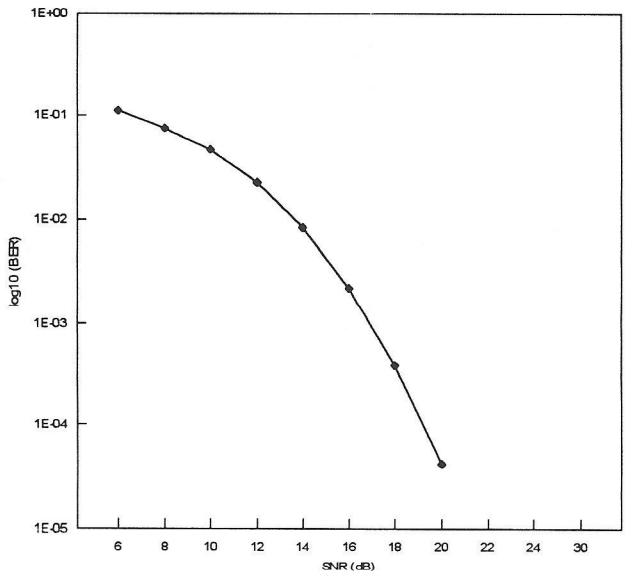

Figure 7.3 - BER performance of RNN equaliser on Channel 1.

8. Radial-basis function equaliser[14][19]

8.1 Radial-basis function node

The Radial-Basis Function (RBF) network uses a different type of node to the standard perceptron. The RBF node shown in figure 8.1 has I inputs, x_i, and for each input the node stores a co-ordinate value, w_i. The node calculates the Euclidean distance in co-ordinate space between the input value and the co-ordinate stored for that input. The sum of the distances is passed through a Gaussian distribution function so that the node fires when the total distance tends to zero. The function of the RBF node can be described as:

$$y_j = \varphi\left[\sum_{i=0}^{I} \|x_i - w_i\|\right] \tag{8.1}$$

where the activation function is:

$$\varphi[v(n)] = \exp\left[\frac{-(v(n))^2}{2\sigma_w^2}\right] \tag{8.2}$$

σ_w represents the width of the Gaussian function. Equations (8.1) and (8.2) imply that the RBF node will fire if the co-ordinates presented to the node are close to the co-ordinate stored.

8.2 RBF networks in equalisation

For an RBF network to equalise channel 1, figure 5.2 shows that eight co-ordinates can be presented to a two input equaliser. In the RBF equaliser there is one node given to each possible co-ordinate. Figure 8.2 shows such a structure. For each received pattern one of the eight nodes will fire. All the outputs to the RBF layer are fed to an ADALINE node. The bias of the ADALINE

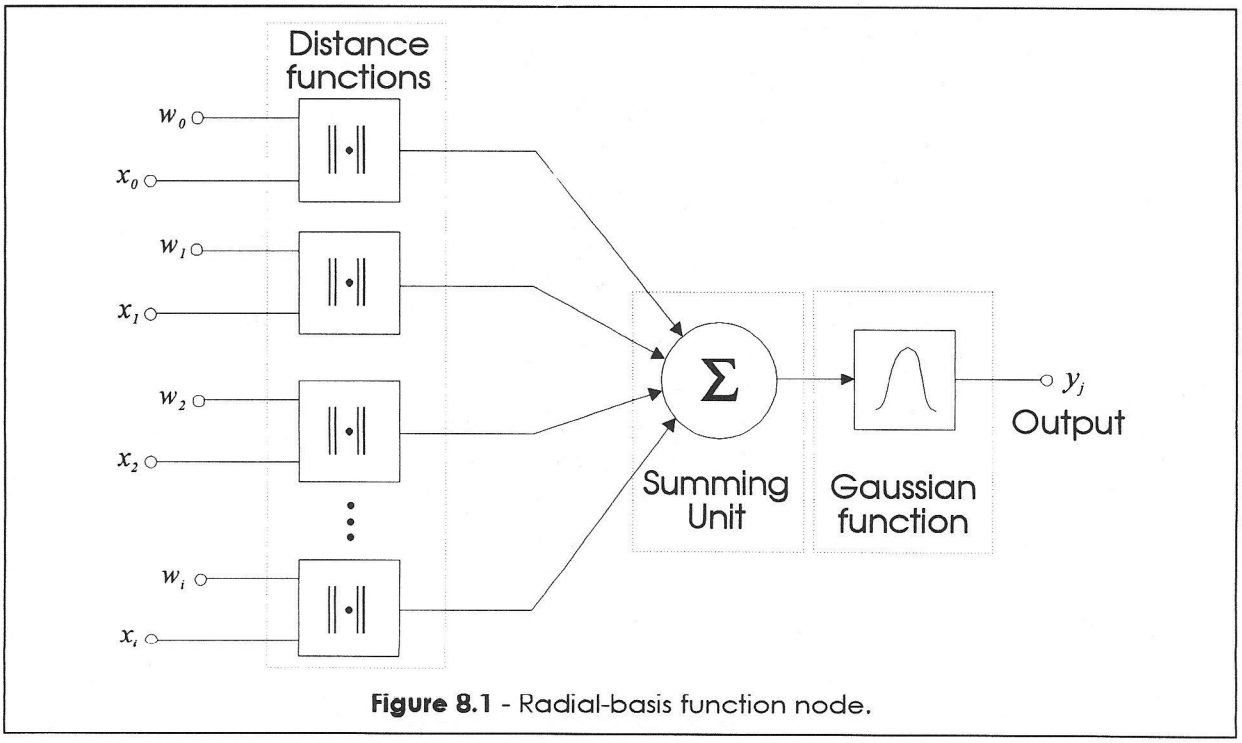

Figure 8.1 - Radial-basis function node.

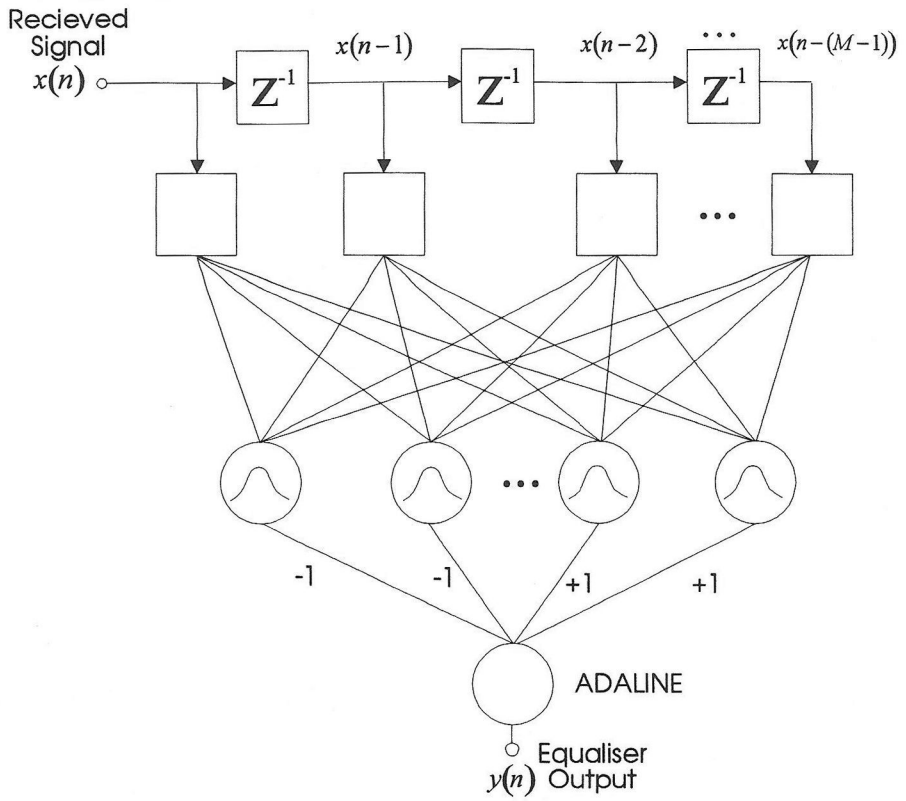

Figure 8.2 - RBF network used as an adaptive equaliser.

node is always zero so the bias node can be ignored.

The RBF network must classify the co-ordinates represented by × as -1.0 and ● as +1.0. Therefore, the weights connected to the RBF nodes designated to detect × co-ordinates are set to -1.0. The weights connected to the ● are set to +1.0. Only one node should fire at a time, so the network output will be close to +1.0 or -1.0.

The RBF network is trained by updating the co-ordinates, or centres, that the RBF nodes store. By using the past record of desired signal data, it can be calculated which node should be firing. The co-ordinates on the inputs to the network are then used to provide a mean centre for that node. This averaging process, after time, will cancel out the effect of the Gaussian noise. After training each node will have its centre on its designated co-ordinate.

8.3 RBF equaliser simulation

Due to the nature of the training, there is no error signal recorded for the RBF equaliser and only the BER test can be performed. Also, because the RBF network does not require any signal to be fed through the network during training only the averaging process is performed. This makes the computational complexity during training low. When training ends, data is fed through the network and the number of distance functions that are performed does make the network slower and restricts the maximum transmission speed.

Figure 8.3 shows the BERs for channels 1 and 2 over a range of SNRs. For all test the Gaussian function width was set to 0.125. It can be seen that for channels 1 and 2 the RBF

equaliser performs extremely well. The length of the training preamble can be short due to the nature of the averaging function of the training procedure.

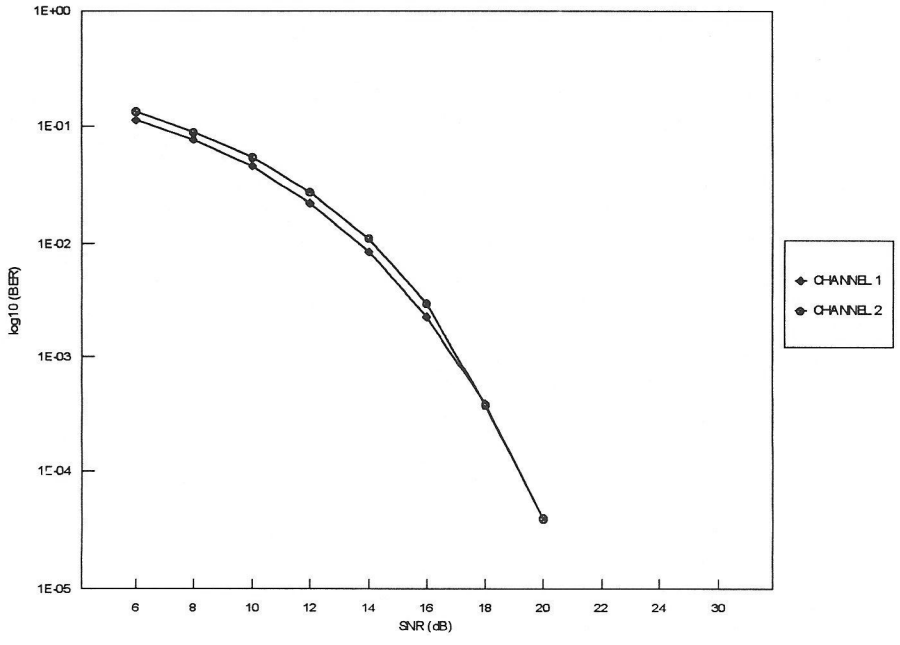

Figure 8.3 - BER performance of RBF equaliser.

9. Assessment of ANN equaliser techniques

Figure 9.1 shows the impulse response of a real life channel model, given by reference [???], that represents a typical telephone line in the British network. All four equalisers were tested on this channel model without noise so that the optimum network parameters could be found. Then they would be tested with these parameters over a range of SNRs.

No optimum parameters could be found for the backpropagation equaliser, whilst keeping the network realistic size, and therefore it was not tested on the real life channel model. The RBF equaliser would also not train on the real life channel model with an acceptable sized structure.

The 10 tap channel model produces 2048 co-ordinate points in two dimensional space, all practically on top of one another. A network would have to contain an extremely large number of nodes to classify this number of points. A large network results in very high computational complexity and stops the network being any practical use as an equaliser.

However both the ADALINE and RNN equalisers did adapt successfully. The LMS equaliser was set-up up with 15 inputs, a learning rate of 0.08 and a delay of 5 bits. The RNN equaliser had 10 inputs, 4 output nodes and a learning rate of 0.003.

Figure 9.2 shows that the convergence of the ADALINE and RNN equalisers are very similar with the RNN achieving a slighter smaller final error. Figure 9.3 shows that the ADALINE network outperformed the RNN equaliser for BERs.

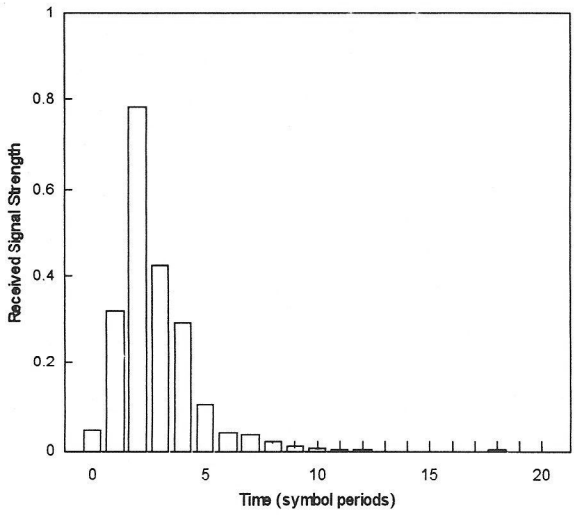

Figure 9.1 - Impulse response of a real life channel

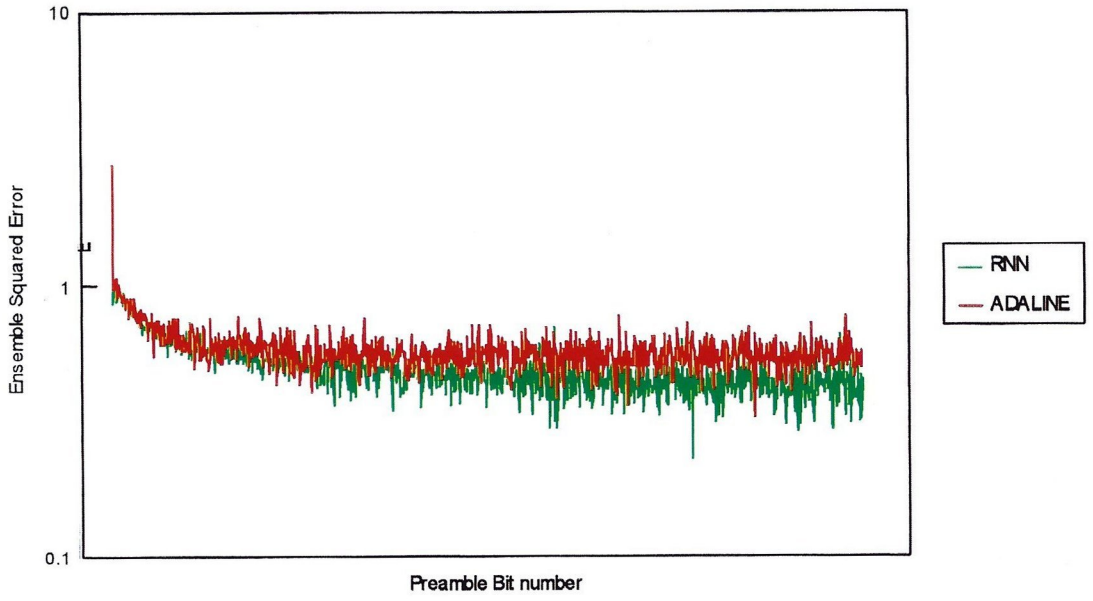

Figure 9.2 - Adaption curves for ADALINE and RNN equalisers for real life channel model.

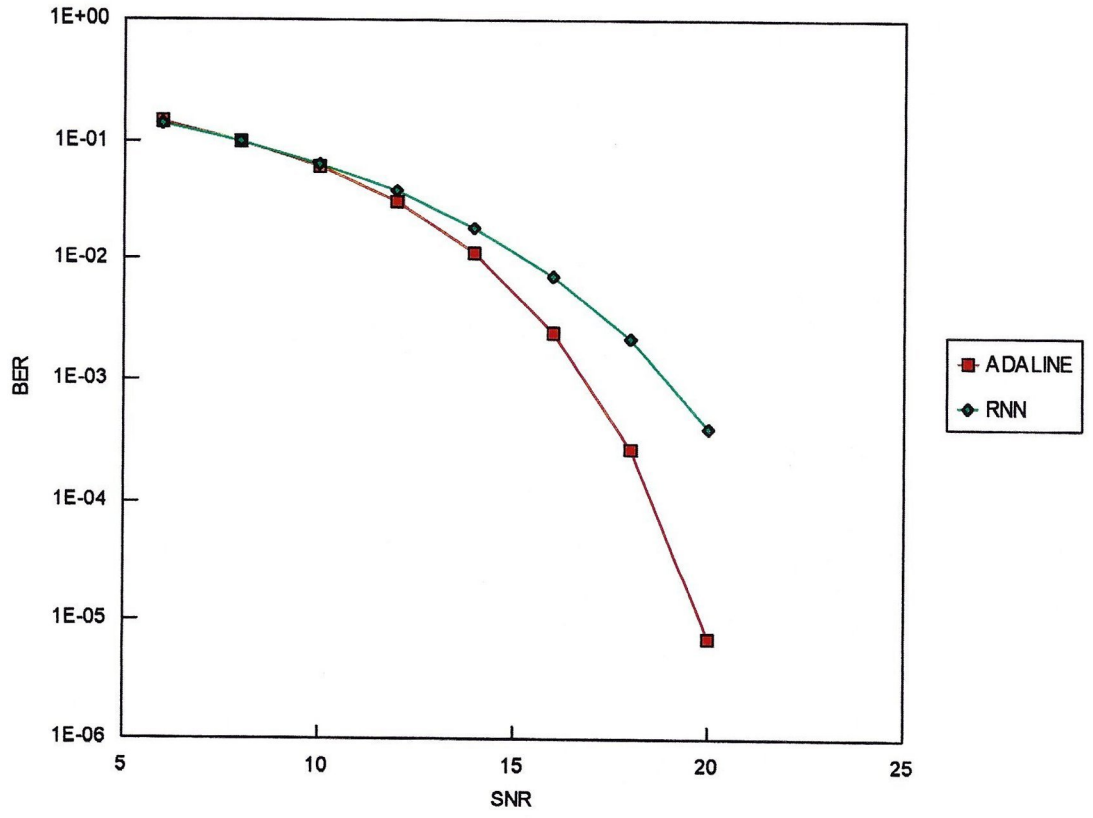

Figure 9.3 - Bit error rate test results for ADALINE and RNN equalisers on real life channel models.

10. Conclusions

The results, or lack of, presented in section 9 suggest that the RBF and backpropagation ANNs are not suitable for equalisation. This was drawn from the fact that the set of inputs co-ordinates were extremley close together and very difficult to separate for a practically sized network. A practically sized network being one that can compete with the already well established ADALINE equaliser in computation speed.

The RNN equaliser showed that it was a useful equaliser, but its high sensitivity to starting parameters and instability made it second best to the ADALINE network. Reference [20] even suggests that RNNs are unsuitable for equalisation.

Thus the three ANNs compared to the ADALINE network showed no advantage in using complex non-linear structures. However, this project only scratched the surface of the field of artificial neural networks and there were various structures not investigated such as the Kohonen self-organising network[21] and FIR MLPs.

Due to the large amount of computing time (coding and simulations) required for these types of investigations, research is was very slow, especially with the computer resources available. A more in depth investigation into the ANNs covered could yield some interesting reasons why the ANNs do not adapt well and give pointers to structures or algorithms that may improve the performance of neural networks in the field of adaptive equalisation.

References

[1] S. U. H. Qureshi, "Adaptive Equalisation," IEEE Communications Magazine, pp 9-16, March 1982.

[2] G. Proakis, "Advances in Equalisation for Intersymbol Interference," Advances in Communication Systems, volume 4, New York Academic Press, 1975.

[3] W. Lucky, "Automatic Equalisation for Digital Communications," Bell Systems Technical Journal, pp. 547-588, April 1965.

[4] W. Lucky, J. Salz and E. J. Weldon Jr., Principles of Data Communication, New York:McGraw-Hill, 1968.

[5] E. Austen, "Decision-Feedback Equalisation for Digital Communication Over Dispersive Channels," M. I. T. Lincoln lab. Technical Report 437, 1967.

[6] J. Gibson, S. Siu and C. F. N. Cowan, "Multi-layer Perceptron Structures Applied to Adaptive Equalisers for Data Communications," ICASSP-89, pp. 1183-1186, 1989.

[7] MATLAB Neural Network Toolbox Manual, Mathworks, 1994.

[8] S. Bendat, A. G. Piersol, Random Data:Analysis and Measurement Procedures, Wiley, 1971.

[9] J. Miller, S. V. Ahamed, Digital Transmission Systems and Networks; Volume 1, Computer Science Press, 1987

[10] E. Ziemer, Introduction to Digital Communication, Macmillan Publising Company, 1992.

[11] Richard P. Lippman, "An introduction to Computing with Neural Nets," IEEE ASSP Magazine, pp 4-21, April 1987.

[12] Drew van Camp, "Neurons for Computers," Scientific American, pp 170-172, September 1992.

[13] Dan Hammerstrom, "Working with Neural Networks," IEEE Spectrum, pp 46-53, July 1993.

[14] S. Haykin, Neural Networks, Macmillan College Publishing, 1994.

[15] B. Widrow, M. A. Lehr, "30 years of adaptive neural networks: Perceptron, MADALINE, and backpropagation," Proceedings of the IEEE 78, pp 1415-1442, 1990.

[16] B. Widrow, S. D. Stearns, Adaptive Signal Procressing, Prentice-Hall, 1985

[17] E. E. Newhall, S. U. H. Querehsi, C. F. Simone, "A Technique for finding Approximate Inverse Systems and Its Application to Equalization," IEEE transactions on Communicatuion Technology, vol. 6., pp1116-1127, December 1971.

[18] G. Kechriotis, E. Zervas, E. S. Manolakos, "Using Recurrent Neural Networks for Adaptive Communication Channel Equalisation," IEEE transactions on Neural Networks, vol. 5,

pp267-278, March 1994.

[19] S. Cheng, B. Mulgrew, P. Grant, "A clustering technique for Digital Communications Channel Equalisation Using Radial-Basis Function Networks," IEEE transactions on Neural Networks, vol. 4, pp570-579, July 1993.

[20] M. J. Bradley, P. Mars, "A Critical Assessment of Recurrent Artificial Neural Networks as Adaptive Equalizers in Digital Communications," IEE Colloquium on Applications of Neural Networks to Signal Processing, 15 December 1994.

[21] T. Kohonen, K. Raivio, O. Simula, O. Ventä, J. Henriksson, "An Adaptive Discrete-signal Dectector based on Self-Organizing maps," IJCNN San Diego, II-249 - II-256, 1989.

[22] S. Haykin, Introduction to Adaptive Filters, Macmillan Publishing Company, 1984.

[23] C. F. N. Cowan, P. M. Grant, Adaptive Filters, Prentice-Hall, 1985.

[24] J. G. Proakis, Digital Communications, McGraw-Hill, 1983.

Appendix A - LMS Derivation[14][22][23][24]

For an ADALINE adaptive equaliser, the error of the output can be calculated by

$$e = d - y \tag{A1}$$

where d is the desired signal and y is the ADALINE output. If the ADALINE has I inputs and I corresponing weights represented by x_i and w_i respectively, the output of the ADALINE is written as

$$y = \sum_{i=0}^{I-1} w_i x_i \tag{A2}$$

The performance of the ADALINE is measured by the cost function, J,

$$J = \frac{1}{2} E\left[e^2\right] \tag{A3}$$

where $E[\cdot]$ is the expectation operator. Combining equations (A1), (A2) and (A3) and using a double summation to represent a squared term gives

$$\begin{aligned} J &= \frac{1}{2} E\left[d^2\right] - E\left[\sum_{i=0}^{I-1} w_i x_i d\right] + \frac{1}{2} E\left[\sum_{i=0}^{I-1}\sum_{j=0}^{I-1} w_i w_j x_i x_j\right] \\ &= \frac{1}{2} E\left[d^2\right] - \sum_{i=0}^{I-1} w_i E\left[x_i d\right] + \frac{1}{2}\sum_{i=0}^{I-1}\sum_{j=0}^{I-1} w_i w_j E\left[x_i x_j\right] \end{aligned}$$

Define

- The mean-square value of the desired response as

$$r_d = E\left[d^2\right]$$

- The cross-correlation fucntion between the desired response and the input as

$$r_{dx}(i) = E\left[d x_i\right]$$

- The autocorrelation of the inputs as

$$r_x(i,j) = E\left[x_i x_j\right]$$

Therefore:

$$J = \frac{1}{2} r_d - \sum_{i=0}^{I-1} w_i r_{dx}(i) + \frac{1}{2}\sum_{i=0}^{I-1}\sum_{j=0}^{I-1} w_i w_j r_x(i,j) \tag{A4}$$

Plotting J against the weight values constitutes the error surface for the ADALINE. The surface is a smooth bowl shape with a minimum J_{min}. The downwards direction of the gradient points towards J_{min}:

$$\nabla_{w_j} J = \frac{\partial J}{\partial w_j} = -r_{dx}(j) + \sum_{i=0}^{I-1} w_i r_x(i,j)$$

Using the gradient descent algorithm, the change to weight i at iteration n is defined as

$$\Delta w_i(n) = -\eta \nabla_{w_i} J(n)$$

The LMS algorithm uses instaneous estimates for $r_x(i,j;n)$ and $r_{xd}(i)$:

$$r_x(i,j;n) = x_i(n)x_j(n)$$
$$r_{dx}(i;n) = x_i(n)d(n)$$

Thus,

$$\begin{aligned} w_i(n+1) &= w_i(n) + \eta\left[x_i(n)d(n) - \sum_j^{I-1} w_j(n)x_i(n)x_j(n)\right] \\ &= w_i(n) + \eta\left[d(n) - \sum_j^{I-1} w_j(n)x_j(n)\right]x_i(n) \\ &= w_i(n) + \eta\left[d(n) - y(n)\right]x_i(n), \qquad i = 0,1,2,\ldots,(I-1) \end{aligned}$$

Appendix B - Program listings

```
/*
        FILENAME:               EQ_FUNCS.H

        DESCRIPTION:    Header file for EQ_FUNCS.C.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/

/* Definitions */

#ifndef M_PI
#define M_PI            3.14159265358979323846 /* Define PI if not already defined */
#endif

#define SIG_A                   1.0
#define SIG_B                   2.0


#define SIGMOID(x)              SIG_A * tanh (SIG_B*x * 0.5)
#define DER_SIGMOID(x) SIG_A * 0.5 * SIG_B * (1.0-x*x)
#define LAYER1                  0
#define LAYER2                  ann.nodes_1
#define LAYER3                  ann.nodes_1 + ann.nodes_2

#define TOTAL_NODES             ann.nodes_1 + ann.nodes_2 + ann.nodes_3
#define TOTAL_WEIGHTS    ann.nodes_0*ann.nodes_1 + ann.nodes_1*ann.nodes_2 +
ann.nodes_2*ann.nodes_3

#define NODES_START             ann.layer_1
#define WEIGHTS_START   NODES_START->weights

#define URAND                   (double)rand()/RAND_MAX

#define RAD_BASIS(x)    exp(-(x*x)/width)

#define KRONECKER(x,y)  (double)(x==y)

/* Structures */

typedef struct
        {
        double  value;                                  /* Value of signal on tap */
        double  weight;                                 /* Value of weight */
        } FIR_MODEL_STR;

typedef struct
        {
        double  sense;
        double  sum;
        } SENSE_STR;

typedef struct
        {
        double                  weight;                 /* Value of weight */
        double                  delta;                  /* Last weight change value */
        double                  delta_sf;               /* The current sum during an epoch used
in the weight change */
        SENSE_STR       far     *sensitivity;   /* Pointer to Sensitivity vector for a RNN */
        } WEIGHT_STR;

typedef struct
        {
        WEIGHT_STR      far     *weights;               /* Pointer to array of weights from this
node to all nodes in previous layer */
        double                  derivative;             /* Nodes error derivative */
        double                  output;                 /* Output of the node */
        double                  counter;                /* Used to calculate the mean in a Radial
basis node */
        } NODE_STR;
```

```
typedef struct
        {
    int                         nodes_0;                /* No of inputs to net */
    int                         nodes_1;                /* No of layer 1 nodes in net */
    int                         nodes_2;                /* No of layer 2 nodes in net */
        int                             nodes_3;                /* No of layer 3 (output) nodes in
net */
        NODE_STR        far     *layer_0;               /* Pointer to input (layer 0) nodes */
        NODE_STR        far     *layer_1;               /* Pointer to hidden (layer 1) nodes */
        NODE_STR        far     *layer_2;       /* Pointer to hidden (layer 2) nodes */
        NODE_STR        far     *layer_3;       /* Pointer to ouput (layer 3) nodes */
        double                  learn_rate;             /* Learning rate */
        double                  momentum;               /* Momentum Factor for backpropagation
algorithm */
    } ANN_STR;

typedef struct
        {
        double                  value;
        double                  desired;
        } TRAIN_STR;

/* Function prototypes */

extern  void            prbs(double far *sequence, int length);
extern  void            shift_right_fir(FIR_MODEL_STR far *filter, int fir_length);
extern  void            ffwd_sig(NODE_STR far *inputs, NODE_STR far *nodes, int no_inputs, int
no_nodes);
extern  void            derivatives(NODE_STR far *layer1, NODE_STR far *layer2, int nodes1, int
nodes2);
extern  void            update_weights_sf(NODE_STR far *layer1, NODE_STR far *layer2, int
nodes1, int nodes2);
extern  void            update_weights_fi(NODE_STR far *layer2, int nodes1, int nodes2, double
lr, double mc);
extern  void            print_weights(NODE_STR far *layer1, NODE_STR far *layer2, int nodes1,
int nodes2);
extern  void            dist(NODE_STR far *inputs, NODE_STR far *nodes, int no_inputs, int
no_nodes, double spread);
extern  void            ffwd_ada(NODE_STR far *inputs, NODE_STR far *nodes, int no_inputs, int
no_nodes);
extern  void            print_dervs(NODE_STR far *layer2, int nodes1);
extern  void            print_deltas(NODE_STR far *layer2, int nodes1, int nodes2);
extern  double          noise(double sig_to_noise);
extern  double          fir_output(FIR_MODEL_STR far *filter, int fir_length);
extern  int                     params(int no_pars, char *par_txt[]);
extern  int             sign(double value);
extern  int             get_state(FIR_MODEL_STR far *data, int length);
extern  int                     pick_rand(int *list, int length);
extern  double          next_pn(int *pn_shift);
extern  void            shift_right_layer(NODE_STR far *layer, int no_nodes);
extern  void            save_doubles(double far *list, int length, double div, char path[],
char *name);
extern  void            fill_rand(NODE_STR far *layer2, int no_nodes1, int no_nodes2);
```

```
/*
        FILENAME:               EQ_FUNCS.C

        DESCRIPTION:    Provides useful functions for adaptive equaliser simulations.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/


/* Include header files */

#include <stdlib.h>
#include <math.h>
#include <conio.h>
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "eq_funcs.h"


int     params(int no_pars, char *par_txt[])
        {
        int     i;
        int             test = 0;
        char    *input_txt;

        if (no_pars > 1)
                {
                input_txt = par_txt[1];
                for (i=0; i<(int)strlen(input_txt); i++)
                        {
                        input_txt[i] = toupper(input_txt[i]);
                        }
                if (!strcmp("TEST", input_txt))
                        {
                        test = 1;
                        }
                }
        return test;
        }

/*
        FUNCTION PROTOTYPE:             double  noise(double sig_to_noise)

        DESCRIPTION:                    Returns a Gaussian distributed number with a variance
defined by the input
                                                parameter sig_to_noise (the SNR).
*/

double noise(double sig_to_noise)
        {
        double  rand1;  /* Holds two uniform random numbers */

/* Generate a uniform random number not equal to zero */

    while ((rand1 = URAND) == 0.0);

/* Return a Gaussian random number */

        return (sqrt(-2.0*pow(10.0, (double)(-sig_to_noise/10.0))*log(rand1))*cos(2.0 * M_PI *
URAND));
        }


/*
        FUNCTION PROTOTYPE:             void prbs(double far *sequence, int length)

        DESCRIPTION:                    Fills the structure a sequence with 1's and -1's with
equal probability of
                                                occurance. The sequence length is defined
by the input parameter length.
*/
```

```
void prbs(double far *sequence, int length)
        {
        int     count;                  /* Counter for a FOR loop */

        for(count=0; count<length; count++)
                {
                if (URAND > 0.5)
                        {
                        sequence[count]=1.0;
                        }
                else
                        {
                        sequence[count]=-1.0;
                        }
                }
        }

/*
        FUNCTION PROTOTYPE:             double fir_output(FIR_MODEL_STR far *filter, int
fir_length)

        DESCRIPTION:                    Sums the product of each value held in the FIR structure
at filter, with its
                                                respective weight. The number of taps in
the filter is given by the input
                                                parameter fir_length.
*/

double fir_output(FIR_MODEL_STR far *filter, int fir_length)
        {
        int i;                          /* Counter for a FOR loop */
        double prod = 0.0;      /* Holds the current sum of products */

        for(i=0; i<fir_length; i++)
                {
                prod += filter[i].value * filter[i].weight;
                }
        return prod;
        }

/*
        FUNCTION PROTOTYPE:             void shift_right_fir(FIR_MODEL_STR far *filter, int
fir_length)

        DESCRIPTION:                    Shifts the values on the delay line of the FIR filter
structure at filter along to the
                                                right one place. The length of the FIR
filter is given by fir_length.
*/

void shift_right_fir(FIR_MODEL_STR far *filter, int fir_length)
        {
        int i;

        for(i=(fir_length-1); i>0; i--)
                {
                filter[i].value = filter[i-1].value;
                }
        }

/*
        FUNCTION PROTOTYPE:             int sign(double value)

        DESCRIPTION:                    Returns the sign (+1 or -1) of the input parameter
value.
*/

int sign(double value)
        {
        int result = 1;

        if (value <0.0)
                {
                result = -1;
                }
```

```
        return result;
        }

/*
        FUNCTION PROTOTYPE:             void update_weights_sf(NODE_STR far *layer2, int nodes1,
int nodes2, double lr, double mc)

        DESCRIPTION:                    Updates weights between layers 1 and 2 during epoch
training with BP alogorithm
*/

void update_weights_sf(NODE_STR far *layer1, NODE_STR far *layer2, int nodes1, int nodes2)
        {
        int             i,j;

        for(i=0; i<nodes2; i++)
                {
                for(j=0; j<nodes1; j++)
                        {
                        layer2[i].weights[j].delta_sf += layer2[i].derivative *
layer1[j].output;
                        }
                }
        }

/*
        FUNCTION PROTOTYPE:             void update_weights_fi(NODE_STR far *layer2, int nodes1,
int nodes2, double lr, double mc)

        DESCRIPTION:                    Updates weights between layers 1 and 2 at end of epoch
with BP alogorithm
*/

void update_weights_fi(NODE_STR far *layer2, int nodes1, int nodes2, double lr, double mc)
        {
        int             i,j;

        for(i=0; i<nodes2; i++)
                {
                for(j=0; j<nodes1; j++)
                        {
                        layer2[i].weights[j].weight += layer2[i].weights[j].delta =
layer2[i].weights[j].delta_sf * lr * (1.0 - mc) +

                                                                mc * layer2[i].weights[j].delta;
                        layer2[i].weights[j].delta_sf = 0.0;
                        }
                }
        }

/*
        FUNCTION PROTOTYPE:             void derivatives(NODE_STR far *layer1, NODE_STR far
*layer2, int nodes1, int nodes2)

        DESCRIPTION:                    Calculates local derivatives for nodes in layer 1
*/

void derivatives(NODE_STR far *layer1, NODE_STR far *layer2, int nodes1, int nodes2)
        {
        int     i,j;
        double  sum;

        for(i=0; i<nodes1; i++)
                {
                sum = 0.0;
                for(j=0; j<nodes2; j++)
                        {
                        sum += layer2[j].weights[i].weight*layer2[j].derivative;
                        }
                layer1[i].derivative = sum * DER_SIGMOID(layer1[i].output);
                }
        }

/*
        FUNCTION PROTOTYPE:             void ffwd_sig(NODE_STR far *inputs, NODE_STR far *nodes,
```

```
int no_inputs, int no_nodes)

        DESCRIPTION:                    Feeds signal in layer 1 through to layer 2 with sigmoid
activation functions
*/

void ffwd_sig(NODE_STR far *inputs, NODE_STR far *nodes, int no_inputs, int no_nodes)
        {
        int             i,j;
        double  input;

        for (i=0; i<no_nodes; i++)
                {
                input = 0.0;
                for(j=0; j<no_inputs; j++)
                        {
                        input += inputs[j].output * nodes[i].weights[j].weight;
                        }
                nodes[i].output =SIGMOID(input);
                }
        }

/*
        FUNCTION PROTOTYPE:             void print_weights(NODE_STR far *layer1, NODE_STR far
*layer2, int nodes1, int nodes2)

        DESCRIPTION:                    Print weights and outputs between layers 1 and 2
*/

void print_weights(NODE_STR far *layer1, NODE_STR far *layer2, int nodes1, int nodes2)
        {

        int i,j;

        for (i=0; i<nodes1; i++)
                {
                for (j=0; j<nodes2; j++)
                        {
                        printf("%+01.5lf ", layer2[j].weights[i].weight);
                        }
                printf(" | %+01.5lf\n", layer1[i].output);
                }
        for (i=0; i<nodes2; i++)
                {
                printf("=========");
                }
        printf("\n");
        for (i=0; i<nodes2; i++)
                {
                printf("%+01.5lf ", layer2[i].output);
                }
    printf("\n\n");
        }

/*
        FUNCTION PROTOTYPE:             void print_dervs(NODE_STR far *layer2, int nodes1)

        DESCRIPTION:                    Prints derivitives in layer 2
*/

void print_dervs(NODE_STR far *layer2, int nodes1)
        {

        int i;

        for (i=0; i<nodes1; i++)
                {
                printf("%+01.5lf ", layer2[i].derivative);
                }
        printf("\n\n");
        }

/*
        FUNCTION PROTOTYPE:             void print_deltas(NODE_STR far *layer2, int nodes1, int
nodes2)
```

```
        DESCRIPTION:                    Prints changes in weights for layer 2.
*/

void print_deltas(NODE_STR far *layer2, int nodes1, int nodes2)
        {

        int i,j;

        for (i=0; i<nodes1; i++)
                {
                for (j=0; j<nodes2; j++)
                        {
                        printf("%+01.5lf ", layer2[j].weights[i].delta);
                        }
                printf("\n");
                }
        printf("\n\n");
        }

/*
        FUNCTION PROTOTYPE:             int get_state(FIR_MODEL_STR far *data, int length)

        DESCRIPTION:                    Work out which node in RBF layer should fire.
*/

int get_state(FIR_MODEL_STR far *data, int length)
        {
        int             state = 0;                              /* State that data is in */
        int             power;                          /* Holds current power weighting */
        int     i;                                              /* Counter in FOR loop */

        power = (int)pow(2, length-1);

        for (i=0; i<length; i++)
                {
                if (data[i].value == 1)
                        {
                        state += power;
                        }
                power /= 2;
                }
        return state;
        }

/*
        FUNCTION PROTOTYPE:             void dist(NODE_STR far *inputs, NODE_STR far *nodes, int
no_inputs, int no_nodes, double width)

        DESCRIPTION:                    Calculates Euclidean distance for layer nodes.
*/

void dist(NODE_STR far *inputs, NODE_STR far *nodes, int no_inputs, int no_nodes, double
width)
        {
        int             i,j;
        double distance;

        for(i=0; i<no_nodes; i++)
                {
                distance = 0.0;
                for(j=0; j<no_inputs; j++)
                        {
                        distance += (nodes[i].weights[j].weight - inputs[j].output) *
(nodes[i].weights[j].weight - inputs[j].output);
                        }
                nodes[i].output = RAD_BASIS(sqrt(distance));
                }
        }

/*
        FUNCTION PROTOTYPE:             void ffwd_ada(NODE_STR far *inputs, NODE_STR far *nodes,
int no_inputs, int no_nodes)

        DESCRIPTION:                    Feed signals forward through an ADALINE layer
*/
```

```
void ffwd_ada(NODE_STR far *inputs, NODE_STR far *nodes, int no_inputs, int no_nodes)
	{
	int		i,j;
	double	input;

	for (i=0; i<no_nodes; i++)
		{
		input = 0.0;
		for(j=0; j<no_inputs; j++)
			{
			input += inputs[j].output * nodes[i].weights[j].weight;
			}
		nodes[i].output =input;
		}
	}


// Ignore

int	pick_rand(int *list, int length)
	{
	int		i;
	int		index, value;

	index = (int)(((double)rand()/RAND_MAX) * (length-1));

	value = list[index];

	for (i=index; i<length-1; i++)
		{
		list[i] = list[i+1];
		}
	return value;
	}

// Ignore

double next_pn(int *pn_shift)
	{
	int	i;
	double	signal;

    signal = (double)(pn_shift[1] ^ pn_shift[4]);

	for(i=4; i>0; i--)
		{
		pn_shift[i] = pn_shift[i-1];
		}

	pn_shift[0] = (int)signal;

	if (signal == 0)
		{
		signal = -1.0;
		}
	return signal;
	}

/*
	FUNCTION PROTOTYPE:		void shift_right_layer(NODE_STR far *layer, int
no_nodes)

	DESCRIPTION:			Shift values in layer one plave to right
*/

void shift_right_layer(NODE_STR far *layer, int no_nodes)
	{
	int	i;

	for(i=no_nodes-1; i>0; i--)
		{
		layer[i].output = layer[i-1].output;
		}
	}

/*
```

```
        FUNCTION PROTOTYPE:             void save_doubles(double far *list, int length, double
div, char path[], char *name

        DESCRIPTION:                    Saves an array of doubles to disk, first dividing by
div.
*/

void save_doubles(double far *list, int length, double div, char path[], char *name)
        {
    FILE        *output_file;
        char    filename[100];
        int     i;

        sprintf(filename, "%s%s", path, name);
        if ((output_file = fopen(filename, "wt")) == NULL)
                {
                printf("File %s cannot be created", name);
                exit(1);
                }
        for (i=0; i<length; i++)
                {
                fprintf(output_file,"%+2.7lf\n", list[i]/div);
                }
        fclose(output_file);
        }

/*
        FUNCTION PROTOTYPE:             void fill_rand(NODE_STR far *layer_2, int nodes_1, int
nodes_2)

        DESCRIPTION:                    Fills weights between layer 1 and layer2 with small
random numbers
*/

void fill_rand(NODE_STR far *layer_2, int nodes_1, int nodes_2)
        {
        int i,j;

        for(i=0; i<nodes_2; i++)
                {
                for(j=0; j<nodes_1; j++)
                        {
                        layer_2[i].weights[j].weight = URAND*0.1;
                        }
                }
    }
```

```
/*
       FILENAME:              LMS.H

       DESCRIPTION:   Header file for LMS.C.

       RESTRICTIONS:  None

       VERSION:               1       14/04/95       Original version.
*/


/* Global variables */

extern ANN_STR                        ann;                  /* Neural network data structure
*/
extern FIR_MODEL_STR  far     *modelline;     /* Pointer to channel model shift register */
extern FIR_MODEL_STR  far     *delayline;     /* Pointer to channel model shift register */
extern double                 far     *errors;              /* Pointer to output error vector
*/
extern double                 far     *error_run;
extern double                         snr;                  /* Signal to noise ratio in dB's
*/
extern double                         pass_threshold;
extern int                                    taps;                 /* Number of equalizer
taps */
extern int                                    m_taps;               /* Number of taps in
channel model FIR filter */
extern int                                    bits;                 /* Length of preamble in
bits */
extern int                                    runs;                 /* Number of trials to run
*/
extern int                                    x,y;                  /* Display co-ordinates */
extern int                                    pn_shift[5];
extern int                                    delay;

/* Function prototypes */
extern void           lms_init(char par_txt[]);
extern void           lms_tidyup(void);
```

```
/*
        FILENAME:               LMS_INIT.H

        DESCRIPTION:    Initialisation file for LMS equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include "eq_funcs.h"
#include "lms.h"


ANN_STR                                 ann;                    /* Neural network data structure
*/
FIR_MODEL_STR  far              *modelline;     /* Pointer to channel model shift register */
FIR_MODEL_STR  far              *delayline;     /* Pointer to channel model shift register */
double                  far             *errors;                /* Pointer to output error vector
*/
double                  far             *error_run;
double                                  snr;                    /* Signal to noise ratio in dB's
*/
double                                  pass_threshold;
int                                             taps;                   /* Number of equalizer
taps */
int                                             m_taps;                 /* Number of taps in
channel model FIR filter */
int                                             bits;                   /* Length of preamble in
bits */
int                                             runs;                   /* Number of trials to run
*/
int                                             x,y;                    /* Display co-ordinates */
int                                             delay;          /* How long to delay desired
signal for */
int                                             pn_shift[5] =
                                                        {1, 1, 1, 1, 1};        /* PN sequence
shift register */

void lms_init(char path[])
        {
        FILE                    *consts_file;
        FILE                    *weights_file;
        char                    filename[100];
        int                             i;


/* Display start banner */

        printf("\n");
        _setcursortype(_NOCURSOR);
        textcolor(WHITE);
        textbackground(BLACK);
        highvideo();
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)220);
                }
        textbackground(BLUE);
        cprintf("%c ADALINE EQUALISER SIMULATOR v1.0
%c", (char)221, (char)222);
        cprintf("%c               FINAL YEAR PROJECT by           IIIE 08/04/95
%c", (char)221, (char)222);
        textbackground(BLACK);
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)223);
                }
        textbackground(BLUE);
        y=wherey()-3;
        x=70;
```

```
/* Load up simulation constants */

        sprintf(filename, "%sLMS_CON.TXT", path);
        if ((consts_file = fopen(filename, "rt")) != NULL)
                {
                fscanf(consts_file, "%d%d%lf%d%d%d%lf", &m_taps, &bits, &snr, &runs, &delay,
&ann.nodes_0, &ann.learn_rate);
                fclose(consts_file);
                }
        else
                {
                printf("\n\nFile %s not found", filename);
                exit(1);
                }

        ann.nodes_1 = 1;

/* Allocate memory for channel model FIR filter */

        if ((modelline = (FIR_MODEL_STR far *)calloc(m_taps, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on modelline");
                exit(1);
                }

/* Load in channel model weights */

        sprintf(filename, "%sWEIGHTS.TXT", path);
        if ((weights_file = fopen(filename, "rt")) != NULL)
                {
                for(i=0; i<m_taps; i++)
                        {
                        fscanf(weights_file, "%lf\n", &modelline[i].weight);
                        }
                fclose(weights_file);
                }
        else
                {
                printf("\n\nFile %s not found", filename);
                exit(1);
                }

/* Allocate memory for ann layers */

        if ((ann.layer_0 = (NODE_STR  far *)calloc(ann.nodes_0+1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
                exit(1);
                }

        if ((ann.layer_1 = (NODE_STR  far *)calloc(1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
                exit(1);
                }

/* Allocate each node a weight vector */

        if ((ann.layer_1[0].weights = (WEIGHT_STR far *)calloc(ann.nodes_0+1,
sizeof(WEIGHT_STR))) == NULL)
                {
                printf("Calloc failiure on weights\n");
                exit(1);
                }

/* Allocate memory for square error record */

        if ((errors = (double  far *)calloc(bits, sizeof(double))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }

/* Allocate memory for square error */
```

```
        if ((error_run = (double  far *)calloc(bits, sizeof(double))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }

/* Allocate memory for desired signal delay line */

        if ((delayline = (FIR_MODEL_STR far *)calloc(delay+1, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on delayline");
                exit(1);
                }

        }

void lms_tidyup(void)
        {

/* Free all allocated memory */


        free((void *)ann.layer_1[0].weights);
        free((void *)modelline);
        free((void *)delayline);
        free((void *)errors);
        free((void *)ann.layer_0);
    free((void *)ann.layer_1);

/* Restore screen */

        textcolor(WHITE);
        textbackground(BLACK);
    lowvideo();
        _setcursortype(_NORMALCURSOR);
    printf("\n\n");
        }
```

```
/*
        FILENAME:               LMS.C

        DESCRIPTION:    Main program for LMS equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <time.h>
#include <math.h>
#include <ctype.h>
#include <string.h>
#include "eq_funcs.h"
#include "lms.h"

void main(int no_pars, char *par_txt[])
        {
        double          error;                          /* Current error of equaliser's output */
        double                  ber = 0;                                /* Holds the bit error
rate */
        double                  signal;                                 /* Next bit to transmit */
        double                  failiures = 0;
        double                  mean_err;
        time_t                  t;                                              /* Variable used to
seed random number generator */
        char                    path[100];                              /* Holds path that .exe
file is in */
        int                     no_errors;                              /* Holds the number of
errors occurred */
        int                             test;                                   /* Holds whether to
find BER for equaliser */
        int                     i, j, bit_no;                   /* Counters in FOR loops */
        int                             run_num;                 /* Current trial number */
        int                             delay_count;                    /* Counts down whilst
delaying equalisation */
        int                             fail;

/* Get path name */

        strncpy(path, par_txt[0], strlen(par_txt[0]) - strlen("LMS.EXE"));
        path[strlen(par_txt[0]) - strlen("LMS.EXE")] = '\0';

/* Initialize LMS equalizer */

        lms_init(path);

/* Find out whether to test the BER of equaliser after training */

        test = params(no_pars, par_txt);

/* Start simulations */

        for (run_num=0; run_num<runs; run_num++)
                {
                gotoxy(x, y);
                printf("TRIAL:%03d",run_num+1);

        fail = 1;

        while(fail)
                {

/* Seed the random number generator */

                time(&t);
                        srand((unsigned)t);

/* Reset delay lines and equalizer weights */

                        for (i=0; i<m_taps; i++)
```

```
                                {
                                modelline[i].value = -1.0;
                                }

                        for (i=0; i<ann.nodes_0; i++)
                                {
                                ann.layer_0[i].output  = -1.0;
                                }

/* Initialise the network */

                        fill_rand(ann.layer_1, ann.nodes_0+ann.nodes_1+1, ann.nodes_1);

/* Set start bit */

                        signal = 1.0;

/* Remember how long to delay the desired response */

                        delay_count = delay;

                        gotoxy(x-10, y);
                        printf("TRAINING ");
                        mean_err = 0.0;

                        for (bit_no = 0; bit_no<bits; bit_no++)
                                {

/* Add next preamble bit into channel model */

                                delayline[0].value = modelline[0].value = signal;

/* Calculate signal and add to equaliser shift register */

                                ann.layer_0[0].output = fir_output(modelline, m_taps) +
noise(snr);

/* Only start equalisation if desired response has been delayed enough */

                                if (delay_count == 0)
                                        {

/* Feed inputs through net */

                                        ffwd_ada(ann.layer_0, ann.layer_1, ann.nodes_0+1,
ann.nodes_1);

/* Calculate error and store square error */

                                        error = delayline[delay].value - ann.layer_1[0].output;
                                        error_run[bit_no-delay] = error*error;
                                        if(bit_no>=bits-400)
                                                {
                                                mean_err += error*error;
                                                }

/* Adjust equalizer weights according to LMS algorithm */

                                        for(j=0; j<ann.nodes_0+1; j++)
                                                {
                                                ann.layer_1[0].weights[j].weight += 2.0 *
ann.learn_rate *  error * ann.layer_0[j].output;
                                                }
                                        }
                                else
                                        {

/* Equalization has not begun so decrease delay counter */

                                        delay_count--;
                                        }

/* Shift channel model and equalizer shift registers one place to the right */

                        signal = next_pn(pn_shift);
                                shift_right_layer(ann.layer_0, ann.nodes_0);
```

```
                        shift_right_fir(modelline, m_taps);
                        shift_right_fir(delayline, delay+1);
                        }
                if ((mean_err/400) < 0.6)
                        {
                        fail = 0;
                        gotoxy(x - 4, y+1);
                        printf("%1.3lf PASSED", mean_err/400);
                        }
                else
                        {
                        gotoxy(x-4, y+1);
                        printf("%1.3lf FAILED", mean_err/400);
//                      putch((char)7);
         failiures++;
                        }
                }
        for(i=0; i<bits-delay; i++)
                {
                errors[i] += error_run[i];
                }

/* Now work out BER by testing over 10000 bits */

        if (test)
                {
                gotoxy(x-10, y);
                printf("TESTING   ");

/* Reset error counter */

                no_errors = 0;

                for(i=0; i<10000; i++)
                        {

/* Add next preamble bit into channel model */

                        delayline[0].value = modelline[0].value = sign((2.0*URAND)-1.0);

/* Calculate signal and add to equaliser shift register */

                        ann.layer_0[0].output = fir_output(modelline, m_taps) +
noise(snr);

/* Feed inputs through net */

                        ffwd_ada(ann.layer_0, ann.layer_1, ann.nodes_0+1, ann.nodes_1);

/* Calculate output of equalizer and then work out its error */

                        if (sign(ann.layer_1[0].output) != delayline[delay].value)
                                {
                                no_errors++;
                                }

/* Shift channel model and equalizer shift registers one place to the right */

                        shift_right_layer(ann.layer_0, ann.nodes_0);
                        shift_right_fir(modelline, m_taps);
                        shift_right_fir(delayline, delay+1);
                        }
                gotoxy(x-25, y);
                printf("ERRORS:%05d", no_errors);
                ber += (double)no_errors/10000;
                }
        }
/* Store mean errors in a text file */

        save_doubles(errors, bits-delay, runs, path, "ERRORS.TXT");

/* Store mean bit error rate in a text file */

        save_doubles(&ber, 1, runs, path, "BER.TXT");

/* Store mean number of failiures in a text file */
```

```
        save_doubles(&failiures, 1, runs, path, "FAIL.TXT");

/* Clean up memory */

        lms_tidyup();
        exit(0);
        }
```

```
/*
        FILENAME:               RNNBC.H

        DESCRIPTION:    Header file for RNNBC.C.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/

extern  ANN_STR                         ann;                    /* Neural network data structure
*/
extern  FIR_MODEL_STR  far      *modelline;     /* Pointer to channel model shift register */
extern  FIR_MODEL_STR  far      *delayline;     /* Pointer to channel model shift register */
extern  double                  far     *errors;                /* Pointer to output error vector
*/
extern  double                  far     *error_run;             /* Pointer to output error vector
*/
extern  double                          snr;                    /* Signal to noise ratio in dB's
*/
extern  int                                     taps;                   /* Number of equalizer
taps */
extern  int                                     m_taps;                 /* Number of taps in
channel model FIR filter */
extern  int                                     bits;                   /* Length of preamble in
bits */
extern  int                                     runs;                   /* Number of trials to run
*/
extern  int                                     x,y;                    /* Display co-ordinates */
extern  int                                     pn_shift[5];
extern  int                                     delay;

extern  void    rnn_init(char par_txt[]);
extern  void    rnn_tidyup(void);
```

```
/*
        FILENAME:               RNN_INIT.H

        DESCRIPTION:    Initialisation file for RNN equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "eq_funcs.h"
#include "rnnbc.h"

/* Global Variables */

ANN_STR                                 ann;                    /* Neural network data structure
*/
FIR_MODEL_STR  far              *modelline;     /* Pointer to channel model shift register */
FIR_MODEL_STR  far              *delayline;     /* Pointer to channel model shift register */
double                  far             *errors;                /* Pointer to output error vector
*/
double                  far             *error_run;             /* Pointer to output error vector
*/
double                                  snr;                    /* Signal to noise ratio in dB's
*/
int                                             taps;                   /* Number of equalizer
taps */
int                                             m_taps;                 /* Number of taps in
channel model FIR filter */
int                                             bits;                   /* Length of preamble in
bits */
int                                             runs;                   /* Number of trials to run
*/
int                                             x,y;                    /* Display co-ordinates */
int                                             delay;          /* How long to delay desired
signal for */
int                                             pn_shift[5] =
                                                        {1, 1, 1, 1, 1};        /* PN sequence
shift register */

void rnn_init(char path[])
        {
        FILE                    *consts_file;           /* Pointer to the input constants file */
        FILE                    *weights_file;          /* Pointer to the input channel model
weights file */
        char                    filename[100];          /* Temporary store for filename to open
*/
        int                             i,j;                            /* Counters for FOR loops
*/

/* Display start banner */

        printf("\n");
        _setcursortype(_NOCURSOR);
        textcolor(WHITE);
        textbackground(BLACK);
        highvideo();
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)220);
                }
        textbackground(BLUE);
        cprintf("%c RECURRENT NET EQUALIZER SIMULATOR v1.0
%c", (char)221, (char)222);
        cprintf("%c               FINAL YEAR PROJECT by           IIIE 08/04/95
%c", (char)221, (char)222);
        textbackground(BLACK);
        for (i=0; i<80; i++)
                {
```

```
                cprintf("%c",(char)223);
                }
        textbackground(BLUE);
        y=wherey()-3;
        x=70;

/* Load up simulation constants */

        sprintf(filename, "%sRNN_CON.TXT", path);
        if ((consts_file = fopen(filename, "rt")) != NULL)
                {
                fscanf(consts_file, "%d%d%lf%d%d%d%d%lf", &m_taps, &bits, &snr, &runs, &delay,
&ann.nodes_0, &ann.nodes_1, &ann.learn_rate);
                fclose(consts_file);
                }
        else
                {
                printf("\n\nFile %s not found\n", filename);
                exit(1);
                }

/* Allocate memory for channel model FIR filter */

        if ((modelline = (FIR_MODEL_STR far *)calloc(m_taps, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on modelline");
                exit(1);
                }

/* Load in channel model weights */

        sprintf(filename, "%sWEIGHTS.TXT", path);
        if ((weights_file = fopen(filename, "rt")) != NULL)
                {
                for(i=0; i<m_taps; i++)
                        {
                        fscanf(weights_file, "%lf\n", &modelline[i].weight);
                        }
                fclose(weights_file);
                }
        else
                {
                printf("File %s not found", filename);
                exit(1);
                }

/* Allocate memory for ann layers */

        if ((ann.layer_0 = (NODE_STR  far *)calloc(ann.nodes_0 + ann.nodes_1 + 1,
sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
                exit(1);
                }

        if ((ann.layer_1 = (NODE_STR  far *)calloc(ann.nodes_1, sizeof(NODE_STR))) == NULL)
                {
                printf("\n\nCalloc failiure on nodes\n");
                exit(1);
                }

/* Allocate each node a weight vector and each weight a sensitivity vector */

        for(i=0; i<ann.nodes_1; i++)
                {
                if ((ann.layer_1[i].weights = (WEIGHT_STR far
*)calloc(ann.nodes_0+ann.nodes_1+1, sizeof(WEIGHT_STR))) == NULL)
                        {
                        printf("Calloc failiure on weights\n");
                        exit(1);
                        }
                for(j=0; j<ann.nodes_0+ann.nodes_1+1; j++)
                        {
                        if ((ann.layer_1[i].weights[j].sensitivity = (SENSE_STR far
*)calloc(ann.nodes_1, sizeof(SENSE_STR))) == NULL)
                                {
```

```
                        printf("Calloc failiure on nodes\n");
                        exit(1);
                        }
                    }
                }

/* Allocate memory for square error record */

        if ((errors = (double  far *)calloc(bits, sizeof(double))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }

/* Allocate memory for square error */

        if ((error_run = (double  far *)calloc(bits, sizeof(double))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }

/* Allocate memory for desired signal delay line */

        if ((delayline = (FIR_MODEL_STR far *)calloc(delay+1, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on delayline");
                exit(1);
                }
        }

void rnn_tidyup(void)
        {
        int i,j;                /* Counters in FOR loops */

/* Free all allocated memory */


        for (i=0; i<ann.nodes_1; i++)
                {
                for(j=0; j<ann.nodes_0+ann.nodes_1; j++)
                        {
                        free((void *)ann.layer_1[i].weights[j].sensitivity);
                        }
                free((void *)ann.layer_1[i].weights);
                }

        free((void *)modelline);
        free((void *)delayline);
        free((void *)errors);
        free((void *)error_run);
        free((void *)ann.layer_0);
    free((void *)ann.layer_1);

/* Restore screen */

        textcolor(WHITE);
        textbackground(BLACK);
    lowvideo();
        _setcursortype(_NORMALCURSOR);
    printf("\n\n");
        }
```

```
/*
        FILENAME:               RNN.C

        DESCRIPTION:    Main program for RNN equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <string.h>
#include <ctype.h>
#include <time.h>
#include <math.h>
#include "eq_funcs.h"
#include "rnnbc.h"

void main(int no_pars, char *par_txt[])
        {
        double          error;                          /* Current error of equaliser's output */
        double                  ber = 0;                                /* Holds the bit error
rate */
        double                  signal;                                 /* Next bit to transmit */
        double                  failiures = 0;
        double                  mean_err;
        time_t                  t;                                              /* Variable used to
seed random number generator */
        char                    path[100];                              /* Holds path that .exe
file is in */
        int                     no_errors;                              /* Holds the number of
errors occurred */
        int                             test;                                   /* Holds whether to
find BER for equaliser */
        int                     i, j, k, l, bit_no;             /* Counters in FOR loops */
        int                             run_num;                /* Current trial number */
        int                             delay_count;                    /* Counts down whilst
delaying equalisation */
        int                             fail;

        /* Get path name */

        strncpy(path, par_txt[0], strlen(par_txt[0]) - strlen("LMS.EXE"));
        path[strlen(par_txt[0]) - strlen("LMS.EXE")] = '\0';

/* Initialise data */

        rnn_init(path);

/* Find out whether to test the BER of equaliser after training */

        test = params(no_pars, par_txt);

/* Start simulations */

        for (run_num=0; run_num<runs; run_num++)
                {
                gotoxy(x, y);
                printf("TRIAL:%03d",run_num+1);

         fail = 1;

         while(fail)
                {

/* Seed the random number generator */

                time(&t);
                        srand((unsigned)t);

/* Reset channel model delay line*/

                        for (i=0; i<m_taps; i++)
```

```
                        {
                        modelline[i].value = -1.0;
                        }

                for(k=0; k<ann.nodes_1; k++)
                        {
                        for(l=0; l<ann.nodes_0+ann.nodes_1+1; l++)
                                {
                                for(j=0; j<ann.nodes_1; j++)
                                        {
                                        ann.layer_1[k].weights[l].sensitivity[j].sense =
0.0;
                                        }
                                }
                        }

/* Initialise the network */

                fill_rand(ann.layer_1, ann.nodes_0+ann.nodes_1+1, ann.nodes_1);

/* Set bias node */

                ann.layer_0[ann.nodes_0].output = -1.0;

/* Set start bit */

                signal = 1.0;

/* Remember how long to delay the desired response */

                delay_count = delay;

                gotoxy(x-10, y);
                printf("TRAINING ");
                mean_err = 0.0;

                for (bit_no = 0; bit_no<bits; bit_no++)
                        {

        /* Add next preamble bit into channel model */

                        delayline[0].value = modelline[0].value = signal;

        /* Calculate signal and add to equaliser shift register */

                        ann.layer_0[0].output = fir_output(modelline, m_taps) +
noise(snr);

        /* Only start equalisation if desired response has been delayed enough */

                        if (delay_count == 0)
                                {
        /* Copy delayed outputs to inputs */

                                for(j=0; j<ann.nodes_1; j++)
                                        {
                                        ann.layer_0[ann.nodes_0+1+j].output =
ann.layer_1[j].output;
                                        }

        /* Feed inputs through net */

                                ffwd_sig(ann.layer_0, ann.layer_1, ann.nodes_0+1,
ann.nodes_1);

        /* Calculate error and store square error */

                                error = delayline[delay].value - ann.layer_1[0].output;
                                error_run[bit_no-delay] = error*error;
                                if(bit_no>=bits-400)
                                        {
                                        mean_err += error*error;
                                        }

        /* Calculate sensitivities */
```

```
                                        for(k=0; k<ann.nodes_1; k++)
                                                {
                                                for(l=0; l<ann.nodes_0+ann.nodes_1+1; l++)
                                                        {
                                                        for(j=0; j<ann.nodes_1; j++)
                                                                {

        ann.layer_1[k].weights[l].sensitivity[j].sum = 0.0;
                                                                for(i=ann.nodes_0+1;
i<ann.nodes_0+1+ann.nodes_1; i++)
                                                                        {

        ann.layer_1[k].weights[l].sensitivity[j].sum += (ann.layer_1[j].weights[i].weight *
ann.layer_1[k].weights[l].sensitivity[i-ann.nodes_0-1].sense +
KRONECKER(k,j)*ann.layer_0[l].output);
                                                                        }
                                                                }
                                                        }
                                                }

                                        for(k=0; k<ann.nodes_1; k++)
                                                {
                                                for(l=0; l<ann.nodes_0+ann.nodes_1+1; l++)
                                                        {
                                                        for(j=0; j<ann.nodes_1; j++)
                                                                {

        ann.layer_1[k].weights[l].sensitivity[j].sense = DER_SIGMOID(ann.layer_1[j].output) *
(ann.layer_1[k].weights[l].sensitivity[j].sum);
                                                                }
                                                        }
                                                }

        /* Update weights */

                                        for(k=0; k<ann.nodes_1; k++)
                                                {
                                                for(l=0; l<ann.nodes_1+ann.nodes_0+1; l++)
                                                        {
                                                        ann.layer_1[k].weights[l].weight +=
ann.learn_rate * error * ann.layer_1[k].weights[l].sensitivity[0].sense;
                                                        }
                                                }
                                }
                                        else
                                                {

        /* Equalization has not begun so decrease delay counter */

                                                delay_count--;
                                                }

        /* Shift channel model and equalizer shift registers one place to the right */

                        signal = next_pn(pn_shift);
                                        shift_right_layer(ann.layer_0, ann.nodes_0);
                                        shift_right_fir(modelline, m_taps);
                                        shift_right_fir(delayline, delay+1);
                                        }
                                if ((mean_err/400) < 0.6)
                                        {
                                        fail = 0;
                                        gotoxy(x - 4, y+1);
                                        printf("%1.3lf PASSED", mean_err/400);
                                        }
                                else
                                        {
                                        gotoxy(x-4, y+1);
                                        printf("%1.3lf FAILED", mean_err/400);
//                                      putch((char)7);
                        failiures++;
                                        }
                                }

                        for(i=0; i<bits-delay; i++)
                                {
```

```
                        errors[i] += error_run[i];
                        }

/* Now work out BER by testing over 10000 bits */

                if (test)
                        {
                        gotoxy(x-10, y);
                        printf("TESTING   ");

/* Reset error counter */

                        no_errors = 0;

                        for(i=0; i<10000; i++)
                                {

/* Add next preamble bit into channel model */

                                delayline[0].value = modelline[0].value = sign((2.0*URAND)-1.0);

/* Calculate signal and add to equaliser shift register */

                                ann.layer_0[0].output = fir_output(modelline, m_taps) +
noise(snr);

/* Copy delayed outputs to inputs */

                                for(j=0; j<ann.nodes_1; j++)
                                        {
                                        ann.layer_0[ann.nodes_0+1+j].output =
ann.layer_1[j].output;
                                        }

/* Feed inputs through net */

                                ffwd_sig(ann.layer_0, ann.layer_1, ann.nodes_0+1, ann.nodes_1);

/* Calculate output of equalizer and then work out its error */

                                if (sign(ann.layer_1[0].output) != delayline[delay].value)
                                        {
                                        no_errors++;
                                        }

/* Shift channel model and equalizer shift registers one place to the right */

                                shift_right_layer(ann.layer_0, ann.nodes_0);
                                shift_right_fir(modelline, m_taps);
                                shift_right_fir(delayline, delay+1);
                                }
                        gotoxy(x-25, y);
                        printf("ERRORS:%05d", no_errors);
                        ber += (double)no_errors/10000;
                        }
                gotoxy(x-4, y+1);
                printf("              ");
                }

/* Store mean errors in a text file */

        save_doubles(errors, bits-delay, runs, path, "ERRORS.TXT");

/* Store mean bit error rate in a text file */

        save_doubles(&ber, 1, runs, path, "BER.TXT");

/* Store mean number of failiures in a text file */

        save_doubles(&failiures, 1, runs, path, "FAIL.TXT");

/* Clean up memory */

        rnn_tidyup();
        }
```

```
/*
        FILENAME:               BPBC.H

        DESCRIPTION:    Header file for EQ_FUNCS.C.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
*/

extern  ANN_STR                         ann;                    /* Neural network data structure
*/
extern  FIR_MODEL_STR   far     *modelline;     /* Pointer to channel model shift register */
extern  double                  far     *errors;                /* Pointer to output error vector
*/
extern  TRAIN_STR       far     *train_vect;    /* Pointer to set of test vectors */
extern  double                          snr;                    /* Signal to noise ratio in dB's
*/
extern  int                             far     *train_list;
extern  int                                     taps;                   /* Number of equalizer
taps */
extern  int                                     m_taps;                 /* Number of taps in
channel model FIR filter */
extern  int                                     bits;                   /* Length of preamble in
bits */
extern  int                                     runs;                   /* Number of trials to run
*/
extern  int                                     x,y;                    /* Display co-ordinates */
extern  int                                     pn_shift[5];
extern  int                                     epochs;

extern  void    bp_init(void);
extern  void    bp_tidyup(void);
```

```
/*
        FILENAME:               BP_INIT.H

        DESCRIPTION:    Initialisation file for Backprop equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "eq_funcs.h"
#include "bpbc.h"

ANN_STR                                 ann;                    /* Neural network data structure
*/
FIR_MODEL_STR  far              *modelline;     /* Pointer to channel model shift register */
double                  far             *errors;                /* Pointer to output error vector
*/
double                                  snr;                    /* Signal to noise ratio in dB's
*/
TRAIN_STR               far             *train_vect;    /* Pointer to set of test vectors */
int                             far             *train_list;
int                                             taps;                   /* Number of equalizer
taps */
int                                             m_taps;                 /* Number of taps in
channel model FIR filter */
int                                             bits;                   /* Length of preamble in
bits */
int                                             runs;                   /* Number of trials to run
*/
int                                             x,y;                    /* Display co-ordinates */
int                                             pn_shift[5] = {1, 1, 1, 1, 1};          /* PN
sequence shift register */
int                                             epochs;

void bp_init(void)
        {
        FILE                    *consts_file;
        FILE                    *weights_file;
        int                             i;
    WEIGHT_STR far      *weight_ptr;
    NODE_STR   far      *node_ptr;

/* Display start banner */

        #ifndef DEBUG
        printf("\n");
        _setcursortype(_NOCURSOR);
        textcolor(WHITE);
        textbackground(BLACK);
        highvideo();
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)220);
                }
        textbackground(BLUE);
        cprintf("%c BACKPROPAGATION EQUALIZER SIMULATOR v1.0
%c", (char)221, (char)222);
        cprintf("%c               FINAL YEAR PROJECT by           IIIE 23/03/95
%c", (char)221, (char)222);
        textbackground(BLACK);
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)223);
                }
        textbackground(BLUE);
        y=wherey()-3;
        x=70;
```

```
        #else
                textmode(C4350);
        clrscr();
        printf("Initializing BP equalizer simulator - debugging mode\n");
        printf("Opening BP constants file\n");
        #endif


        if ((consts_file = fopen("e:\\temp\\bp_const.txt", "rt")) != NULL)
                {
                fscanf(consts_file, "%d%d%d%lf%d%d%d%d%lf%lf", &m_taps, &bits, &epochs, &snr,
&runs, &ann.nodes_0, &ann.nodes_1, &ann.nodes_2,
                                                                        &ann.learn_rate,
&ann.momentum);
                fclose(consts_file);
                }
        else
                {
                printf("File consts.txt not found\n");
                exit(1);
                }

        ann.nodes_3 = 1;

        #ifdef DEBUG
        printf("No of inputs:%02d\n",ann.nodes_0);
        printf("No of layer 1 nodes:%02d\n",ann.nodes_1);
        printf("No of layer 2 nodes:%02d\n",ann.nodes_2);
        printf("No of layer 3 nodes:%02d\n",ann.nodes_3);
        printf("Learn Rate:%+01.5lf\n",ann.learn_rate);
        printf("Momentum:%+01.5lf\n",ann.momentum);
        printf("No of preamble bits:%d\n", bits);
        printf("No of trials:%d\n", runs);
        printf("Signal to noise ratio:%lf dBs\n",snr);
        printf("Allocating memory for model FIR structure: %d * FIR_MODEL_STR\n", m_taps);
        #endif

        if ((modelline = (FIR_MODEL_STR far *)calloc(m_taps, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on modelline");
                exit(1);
                }

        #ifdef DEBUG
        printf("Opening channel model weights file\nModel weights:");
        #endif

        if ((weights_file = fopen("e:\\temp\\weights.txt", "rt")) != NULL)
                {
                for(i=0; i<m_taps; i++)
                        {
                        fscanf(weights_file, "%lf\n", &modelline[i].weight);
                        #ifdef DEBUG
                        printf("%lf\t",modelline[i].weight);
                        #endif
                        }
                #ifdef DEBUG
                printf("\n");
                #endif
                fclose(weights_file);
                }
        else
                {
                printf("File weights.txt not found");
                exit(1);
                }

        #ifdef DEBUG
        printf("Allocating memory for input nodes: %d * bytes\n", (ann.nodes_0+1) *
sizeof(NODE_STR));
        #endif

        if ((ann.layer_0 = (NODE_STR  far *)calloc(ann.nodes_0+1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
```

```
                exit(1);
                }

        #ifdef DEBUG
        printf("Allocating memory for layer 1 nodes: %d * bytes\n", (ann.nodes_1+1) *
sizeof(NODE_STR));
        #endif

        if ((ann.layer_1 = (NODE_STR  far *)calloc(ann.nodes_1+1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on nodes\n");
                exit(1);
                }

        #ifdef DEBUG
        printf("Allocating memory for layer 2 nodes: %d * bytes\n", (ann.nodes_2+1) *
sizeof(NODE_STR));
        #endif

        if ((ann.layer_2 = (NODE_STR  far *)calloc(ann.nodes_2+1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on nodes\n");
                exit(1);
                }

        #ifdef DEBUG
        printf("Allocating memory for layer 3 nodes: %d * bytes\n",
ann.nodes_3*sizeof(NODE_STR));
        #endif

        if ((ann.layer_3 = (NODE_STR  far *)calloc(ann.nodes_3, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on nodes\n");
                exit(1);
                }

        for(i=0; i<ann.nodes_1; i++)
                {
                #ifdef DEBUG
                printf("Allocating memory for layer 1 weights: %d * bytes\n", (ann.nodes_0+1) *
sizeof(WEIGHT_STR));
                #endif

                if ((ann.layer_1[i].weights = (WEIGHT_STR  far *)calloc(ann.nodes_0+1,
sizeof(WEIGHT_STR))) == NULL)
                        {
                        printf("Calloc failiure on nodes\n");
                        exit(1);
                        }
                }

        for(i=0; i<ann.nodes_2; i++)
                {
                #ifdef DEBUG
                printf("Allocating memory for layer 2 weights: %d * bytes\n", (ann.nodes_1+1) *
sizeof(WEIGHT_STR));
                #endif

                if ((ann.layer_2[i].weights = (WEIGHT_STR  far *)calloc((ann.nodes_1+1),
sizeof(WEIGHT_STR))) == NULL)
                        {
                        printf("Calloc failiure on nodes\n");
                        exit(1);
                        }
                }

        #ifdef DEBUG
        printf("Allocating memory for layer 3 weights: %d * bytes\n", (ann.nodes_2+1) *
sizeof(WEIGHT_STR));
        #endif

        if ((ann.layer_3[0].weights = (WEIGHT_STR  far *)calloc((ann.nodes_2+1),
sizeof(WEIGHT_STR))) == NULL)
                {
                printf("Calloc failiure on nodes\n");
                exit(1);
```

```
                }

        #ifdef DEBUG
        printf("Allocating memory for errors: %d * bytes\n", epochs * sizeof(double));
        #endif

        if ((errors = (double  far *)calloc(epochs, sizeof(double))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }

        #ifdef DEBUG
        printf("Allocating memory for test vector: %d * bytes\n", (bits +
(int)ceil((double)ann.nodes_0/2)) * sizeof(TRAIN_STR));
        #endif

        if ((train_vect = (TRAIN_STR  far *)calloc(bits + (int)ceil((double)ann.nodes_0/2),
sizeof(TRAIN_STR))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }

        #ifdef DEBUG
        printf("Allocating memory for test vector list: %d * bytes\n", bits * sizeof(int));
        #endif

        if ((train_list = (int far *)calloc(bits, sizeof(int))) == NULL)
                {
                printf("Calloc failiure on errors\n");
                exit(1);
                }
        }

void bp_tidyup(void)
        {
        int i;

        free((void *)modelline);
        free((void *)errors);
        free((void *)train_vect);
    free((void *)train_list);
    free((void *)ann.layer_0);
    free((void *)ann.layer_1);
    free((void *)ann.layer_2);
        free((void *)ann.layer_3);

        for (i=0; i<ann.nodes_1; i++)
                {
                free((void *)ann.layer_1[i].weights);
                }

        for (i=0; i<ann.nodes_2; i++)
                {
                free((void *)ann.layer_2[i].weights);
                }
        free((void *)ann.layer_3[0].weights);

        #ifdef DEBUG
        printf("Left BP sim OK.\n\n");
        #endif
        }
```

```
/*
        FILENAME:               BPBC.C

        DESCRIPTION:    Main program for Backprop equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <string.h>
#include <ctype.h>
#include <time.h>
#include <math.h>
#include "eq_funcs.h"
#include "bpbc.h"

void main(int no_pars, char *par_txt[])
        {
        FILE                    *output_file;  /* Pointer to the output file */
        FILE                    *error_file;
        double          error;
        double                  ber = 0;                /* Holds the bit error rate */
        double                  desired;                /* Desired output from equaliser during
testing */
        int                     no_errors;              /* Holds the number of errors occurred */
        int                             test = 0;               /* Holds whether to find BER for
equaliser */
        int                     i, j,k;
        int                             index;
        int                             run_num;
        double                  signal;
        char                    filename[50];
        time_t                  t;
        double                  sum_sqr_err;

/* Initialise data */

        bp_init();

/* Find out whether to test the BER of equaliser after training */

        test = params(no_pars, par_txt);

        #ifdef DEBUG
        if (test)
                {
                printf("BER TESTING REQUESTED\n");
                }
        #endif

/* Start simulations */

        for (run_num=0; run_num<runs; run_num++)
                {

                #ifndef DEBUG
                gotoxy(x, y);
                printf("TRIAL:%03d",run_num+1);
                #endif

                srand((unsigned) time(&t));

/* Reset delay lines and network weights */

                for (i=0; i<m_taps; i++)
                        {
                        modelline[i].value = -1.0;
                        }

/* Set start bit */
```

```
                signal = 1.0;

                #ifndef DEBUG
                gotoxy(x-10, y);
                printf("TRAINING ");
                #else
                printf("Generating Training Sequence\n");
                #endif

                for(i=0; i<(bits + ceil((double)ann.nodes_0/2.0)); i++)
                        {
/* Add next preamble bit into channel model */

                        train_vect[i].desired = modelline[0].value = signal;

/* Calculate signal and add to equalilser shift register */

                        train_vect[i].value = fir_output(modelline, m_taps) + noise(snr);

/* Work out next value in PN sequence */

            signal = pn_shift[1] ^ pn_shift[4];

                        for(j=4; j>0; j--)
                                {
                                pn_shift[j] = pn_shift[j-1];
                                }

                        pn_shift[0] = (int)signal;

                        if (signal == 0)
                                {
                                signal = -1.0;
                                }

/* Shift the channel model FIR filter values along */

                        shift_right_fir(modelline, m_taps);
                        }

        #ifdef DEBUG
                printf("Starting training ...\n");
                while(!kbhit());
                getch();
                clrscr();
                #endif
                                        gotoxy(1, 1);
                        print_weights(ann.layer_1, ann.layer_0, ann.nodes_0+1, ann.nodes_1);
                        print_weights(ann.layer_2, ann.layer_1, ann.nodes_1+1, ann.nodes_2);
                        print_weights(ann.layer_3, ann.layer_2, ann.nodes_2+1, ann.nodes_3);
                        printf("\n");
                        while(!kbhit());
                        getch();

/* Initialise the network */

                for(i=0; i<ann.nodes_1; i++)
                        {
                        for(j=0; j<ann.nodes_0+1; j++)
                                {
                                ann.layer_1[i].weights[j].weight = 0.3* URAND
/*(2.4/ann.nodes_0)*/;
                                ann.layer_1[i].weights[j].delta = 0.0;
                                ann.layer_1[i].weights[j].delta_sf = 0.0;
                                }
                        }

                for(i=0; i<ann.nodes_2; i++)
                        {
                        for(j=0; j<ann.nodes_1+1; j++)
                                {
                                ann.layer_2[i].weights[j].weight = 0.3*URAND
/*(2.4/ann.nodes_1)*/;
                                ann.layer_2[i].weights[j].delta = 0.0;
                                ann.layer_2[i].weights[j].delta_sf = 0.0;
                                }
```

```
                        }

                for(i=0; i<ann.nodes_3; i++)
                        {
                        for(j=0; j<ann.nodes_2+1; j++)
                                {
                                ann.layer_3[i].weights[j].weight = 0.3 * URAND
/*(2.4/ann.nodes_1)*/;
                                ann.layer_3[i].weights[j].delta = 0.0;
                                ann.layer_3[i].weights[j].delta_sf = 0.0;
                                }
                        }

/* Set Bias nodes output */

                ann.layer_0[ann.nodes_0].output = -1.0;
                ann.layer_1[ann.nodes_1].output = -1.0;
                ann.layer_2[ann.nodes_2].output = -1.0;

                        gotoxy(1, 1);
                        print_weights(ann.layer_1, ann.layer_0, ann.nodes_0+1, ann.nodes_1);
                        print_weights(ann.layer_2, ann.layer_1, ann.nodes_1+1, ann.nodes_2);
                        print_weights(ann.layer_3, ann.layer_2, ann.nodes_2+1, ann.nodes_3);
                        printf("\n");
                        while(!kbhit());
                        getch();

/* Start training */

                for(i=0; i<epochs; i++)
                        {
                        sum_sqr_err = 0.0;
                        index =ceil((double)ann.nodes_0/2.0);
                        for(j=0; j<bits; j++)
                                {
                         for(k=0; k<ann.nodes_0; k++)
                                        {
                                        ann.layer_0[k].output = train_vect[index-k].value;
                                        }
                                ffwd_sig(ann.layer_0, ann.layer_1, ann.nodes_0+1, ann.nodes_1);
                                ffwd_sig(ann.layer_1, ann.layer_2, ann.nodes_1+1, ann.nodes_2);
                                ffwd_sig(ann.layer_2, ann.layer_3, ann.nodes_2+1, ann.nodes_3);
                                error = train_vect[index].desired - ann.layer_3[0].output;
                                sum_sqr_err += error*error;
                                ann.layer_3[0].derivative = DER_SIGMOID(ann.layer_3[0].output) *
error;
                                derivatives(ann.layer_2, ann.layer_3, ann.nodes_2, ann.nodes_3);
                                derivatives(ann.layer_1, ann.layer_2, ann.nodes_1, ann.nodes_2);
                                update_weights_sf(ann.layer_2, ann.layer_3, ann.nodes_2+1,
ann.nodes_3);
                                update_weights_sf(ann.layer_1, ann.layer_2, ann.nodes_1+1,
ann.nodes_2);
                                update_weights_sf(ann.layer_0, ann.layer_1, ann.nodes_0+1,
ann.nodes_1);
                                index++;
                                }
                        #ifdef DEBUG
            clrscr();
                        gotoxy(1, 1);
                        print_weights(ann.layer_1, ann.layer_0, ann.nodes_0+1, ann.nodes_1);
                        print_weights(ann.layer_2, ann.layer_1, ann.nodes_1+1, ann.nodes_2);
                        print_weights(ann.layer_3, ann.layer_2, ann.nodes_2+1, ann.nodes_3);
                        printf("\n");
                        while(!kbhit());
                        getch();
                        #endif
                        errors[i] += sum_sqr_err/bits;
                        update_weights_fi(ann.layer_3, ann.nodes_2+1, ann.nodes_3,
ann.learn_rate, ann.momentum);
                        update_weights_fi(ann.layer_2, ann.nodes_1+1, ann.nodes_2,
ann.learn_rate, ann.momentum);
                        update_weights_fi(ann.layer_1, ann.nodes_0+1, ann.nodes_1,
ann.learn_rate, ann.momentum);
                        }

/* Now work out BER by testing over 10000 bits */
```

```
                if (test)
                        {
                        #ifndef DEBUG
                        gotoxy(x-10, y);
                        printf("TESTING   ");
                        #else
                        printf("Starting Testing ...\n");
                        #endif

/* Reset error counter */

                        no_errors = 0;

/* Reset delay lines */

                        for(i=0; i<10000; i++)
                                {

/* Add next preamble bit into channel model */

                                desired = modelline[0].value = sign(2.0*URAND-1.0);

/* Calculate signal and add to equalilzer shift register */

                                ann.layer_0[0].output = fir_output(modelline, m_taps) +
noise(snr);

/* Feed the signals through the 3 layers */

                                ffwd_sig(ann.layer_0, ann.layer_1, ann.nodes_0+1, ann.nodes_1);
                                ffwd_sig(ann.layer_1, ann.layer_2, ann.nodes_1+1, ann.nodes_2);
                                ffwd_sig(ann.layer_2, ann.layer_3, ann.nodes_2+1, ann.nodes_3);

/* Calculate output of equalizer and then work out its error */

                                if (sign(ann.layer_3[0].output) != desired)
                                        {
                                        no_errors++;
                                        }

                                for(j=ann.nodes_0-1; j>0; j--)
                                        {
                                        ann.layer_0[j].output = ann.layer_0[j-1].output;
                                        }
                                shift_right_fir(modelline, m_taps);
                                }
                        #ifndef DEBUG
                        gotoxy(x-25, y);
                        printf("LAST BER:%05d", no_errors);
                        #else
                        printf("LAST BER:%05d\n", no_errors);
                        #endif
                        ber += (double)no_errors/10000;
                        }
                }
        #ifdef DEBUG
        clrscr();
        print_weights(ann.layer_1, ann.layer_0, ann.nodes_0, ann.nodes_1);
        print_weights(ann.layer_2, ann.layer_1, ann.nodes_1, ann.nodes_2);
        print_weights(ann.layer_3, ann.layer_2, ann.nodes_2, ann.nodes_3);
        printf("\n");
        while(!kbhit());
        getch();
        #endif

/* Store mean errors in a text file */

        if ((output_file = fopen("e:\\temp\\errors.txt", "wt")) == NULL)
                {
                printf("File errors.txt cannot be created");
                exit(1);
                }

        for (i=0; i<epochs; i++)
                {
```

```
                fprintf(output_file,"%+2.15lf\n", errors[i]/runs);
                }

        fclose(output_file);

/* Store mean bit error rate in a text file */

        if ((output_file = fopen("e:\\temp\\ber.txt", "wt")) == NULL)
                {
                printf("File ber.txt cannot be created");
                exit(1);
                }


        fprintf(output_file,"%+2.15lf\n", ber/runs);

        fclose(output_file);
        bp_tidyup();

        }
```

```
/*
        FILENAME:               RB.H

        DESCRIPTION:    Header file for RB.C.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
*/

extern  ANN_STR                         ann;                    /* Neural network data structure
*/
extern  FIR_MODEL_STR  far      *modelline;      /* Pointer to channel model shift register */
extern  FIR_MODEL_STR  far      *delayline;             /* Pointer to expected signal delay shift
register */
extern  double                          snr;             /* Value for signal to noise ratio */
extern  int                                     taps;                   /* Number of equalizer
taps */
extern  int                                     m_taps;                 /* Number of taps in
channel model FIR filter */
extern  int                                     bits;                   /* Length of preamble in
bits */
extern  int                                     runs;            /* Number of trials to perform */
extern  int                                     x,y;                    /* Display co-ordinates */
extern  int                                     pn_shift[5];    /* PN sequence shift register */
extern  int                                     memory;

extern  void    rb_init(char path[]);
extern  void    rb_tidyup(void);
```

```
/*
        FILENAME:               RB_INIT.H

        DESCRIPTION:    Initialisation file for RBF equaliser simulator.

        RESTRICTIONS:   None

        VERSION:                1       29/03/95        Original version.
                                2       08/04/95        Remove biases from structures -
just use one extra node in the layer above.
*/
#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#include <math.h>
#include "eq_funcs.h"
#include "rb.h"


ANN_STR                         ann;                    /* Neural network data structure */
FIR_MODEL_STR   far     *modelline;     /* Pointer to channel model shift register */
FIR_MODEL_STR   far     *delayline;             /* Pointer to expected signal delay shift
register */
double                          snr;             /* Value for signal to noise ratio */
int                                     taps;                   /* Number of equalizer taps */
int                                     m_taps;                 /* Number of taps in channel model
FIR filter */
int                                     bits;                   /* Length of preamble in bits */
int                                     runs;            /* Number of trials to perform */
int                                     x,y;                    /* Display co-ordinates */
int                                     pn_shift[5]
                                                = {1, 1, 1, 1, 1};      /* PN sequence shift
register */
int                                     memory;                 /* Number of previous bits to
remember */

void rb_init(char path[])
        {
        FILE                    *consts_file;
        FILE                    *weights_file;
        char                    filename[100];
        int                             i;

/* Display start banner */

        printf("\n");
        _setcursortype(_NOCURSOR);
        textcolor(WHITE);
        textbackground(BLACK);
        highvideo();
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)220);
                }
        textbackground(BLUE);
        cprintf("%c RADIAL-BASIS EQUALISER SIMULATOR v1.0
%c", (char)221, (char)222);
        cprintf("%c                FINAL YEAR PROJECT by           IIIE 23/03/95
%c", (char)221, (char)222);
        textbackground(BLACK);
        for (i=0; i<80; i++)
                {
                cprintf("%c",(char)223);
                }
        textbackground(BLUE);
        y=wherey()-3;
        x=70;


/* Load up simulation constants */

        sprintf(filename, "%sRB_CON.TXT", path);
        if ((consts_file = fopen(filename, "rt")) != NULL)
                {
                fscanf(consts_file, "%d%d%lf%d%d%d%lf", &m_taps, &bits, &snr, &runs,
&ann.nodes_0, &ann.nodes_1, &ann.learn_rate);
```

```
                fclose(consts_file);
                }
        else
                {
                printf("\n\nFile %s not found", filename);
                exit(1);
                }

        ann.nodes_2 = 1;

        if ((modelline = (FIR_MODEL_STR far *)calloc(m_taps, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on modelweights");
                exit(1);
                }

/* Load in channel model weights */

        sprintf(filename, "%sWEIGHTS.TXT", path);
        if ((weights_file = fopen(filename, "rt")) != NULL)
                {
                for(i=0; i<m_taps; i++)
                        {
                        fscanf(weights_file, "%lf\n", &modelline[i].weight);
                        }
                fclose(weights_file);
                }
        else
                {
                printf("\n\nFile %s not found", filename);
                exit(1);
                }

/* Allocate memory for ann layers */

        if ((ann.layer_0 = (NODE_STR  far *)calloc(ann.nodes_0, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
                exit(1);
                }

        if ((ann.layer_1 = (NODE_STR  far *)calloc(ann.nodes_1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
                exit(1);
                }

        if ((ann.layer_2 = (NODE_STR  far *)calloc(1, sizeof(NODE_STR))) == NULL)
                {
                printf("Calloc failiure on input nodes\n");
                exit(1);
                }

/* Allocate each node a weight vector */

        for(i=0; i<ann.nodes_1; i++)
                {
                if ((ann.layer_1[i].weights = (WEIGHT_STR far *)calloc(ann.nodes_0,
sizeof(WEIGHT_STR))) == NULL)
                        {
                        printf("Calloc failiure on weights\n");
                        exit(1);
                        }
                }
        if ((ann.layer_2[0].weights = (WEIGHT_STR far *)calloc(ann.nodes_1,
sizeof(WEIGHT_STR))) == NULL)
                {
                printf("Calloc failiure on weights\n");
                exit(1);
                }

/* Fill ADALINE weights vector */

        for(i=0; i<(ann.nodes_1/2); i++)
                {
                ann.layer_2[0].weights[i].weight = -1.0;
```

```
                }

        for(i=i;i<ann.nodes_1;i++)
                {
                ann.layer_2[0].weights[i].weight = 1.0;
                }

        memory = (int)(log10(ann.nodes_1)/log10(2));

        if ((delayline = (FIR_MODEL_STR far *)calloc(memory, sizeof(FIR_MODEL_STR))) == NULL)
                {
                printf("Calloc failiure on delayline");
                exit(1);
                }
        }

void rb_tidyup(void)
        {
        int i,j;

/* Free all allocated memory */

        for (i=0; i<ann.nodes_1; i++)
                {
                free((void *)ann.layer_1[i].weights);
                }
        free((void *)ann.layer_2[0].weights);
        free((void *)modelline);
        free((void *)delayline);
        free((void *)ann.layer_0);
    free((void *)ann.layer_1);
    free((void *)ann.layer_2);

/* Restore screen */

        textcolor(WHITE);
        textbackground(BLACK);
    lowvideo();
        _setcursortype(_NORMALCURSOR);
    printf("\n\n");
        }
```

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <time.h>
#include <math.h>
#include <ctype.h>
#include <string.h>
#include "eq_funcs.h"
#include "rb.h"

void main(int no_pars, char *par_txt[])
        {
        double                  ber = 0;                /* Holds the bit error rate */
    double                      signal;
        double                  output;
        time_t                  t;                              /* Variable used to seed random
number generator */
        char                    path[100];              /* Holds path that .exe file is in */
        int                     no_errors;              /* Holds the number of errors occurred */
        int                             test = 0;               /* Holds whether to find BER for
equaliser */
        int                             index;                  /* Holds which state the output
should be in */
        int                             i, j;                   /* Counters for FOR loops */
        int                             run_num;                /* Holds the current trial number
*/
        int                             delay_count;    /* Counts down whilst delaying
equalisation */

/* Get path name */

        strncpy(path, par_txt[0], strlen(par_txt[0]) - strlen("RB.EXE"));
        path[strlen(par_txt[0]) - strlen("RB.EXE")] = '\0';

/* Initialize LMS equalizer */

        rb_init(path);

/* Find out whether to test the BER of equaliser after training */

        test = params(no_pars, par_txt);

        for (run_num=0; run_num<runs; run_num++)
                {
                gotoxy(x, y);
                printf("TRIAL:%03d",run_num+1);

/* Seed the random number generator */

        time(&t);
                srand((unsigned)t);

/* Reset delay lines and equalizer weights */

                for(i=0; i<ann.nodes_1; i++)
                        {
                        for(j=0; j<ann.nodes_0; j++)
                                {
                                ann.layer_1[i].weights[j].weight = 0.0;
                                }
                        }

                for (i=0; i<m_taps; i++)
                        {
                        modelline[i].value = -1.0;
                        }

                for (i=0; i<ann.nodes_0; i++)
                        {
                        ann.layer_0[i].output  = -1.0;
                        }

                for (i=0; i<ann.nodes_1; i++)
                        {
                        ann.layer_1[i].counter  = 0;
                        }
```

```
/* Set start bit */

                signal = 1.0;

/* Remember how long to delay the desired response */

                delay_count = ann.nodes_0;
                gotoxy(x-10, y);
                printf("TRAINING ");

                for (bit_no = 0; bit_no<bits; bit_no++)
                        {

/* Add next preamble bit into channel model */

                        delayline[0].value = modelline[0].value = signal;

/* Calculate signal and add to equaliser shift register */

                        ann.layer_0[0].output = fir_output(modelline, m_taps) + noise(snr);

/* Only start equalization if desired response has been delayed enough */

                        if (delay_count == 0)
                                {

/* Get state that the output should be in */

                                index = get_state(delayline, memory);

/* Update the mean of the centre given to that state */

                                for(j=0; j<ann.nodes_0; j++)
                                        {
                                        ann.layer_1[index].weights[j].weight *=
ann.layer_1[index].counter;
                                        ann.layer_1[index].weights[j].weight +=
ann.layer_0[j].output;
                                        }

                ann.layer_1[index].counter++;

                                for(j=0; j<ann.nodes_0; j++)
                                        {
                                        ann.layer_1[index].weights[j].weight /=
ann.layer_1[index].counter;
                        }
                                }
                        else
                                {

/* Equalization has not begun so decrease delay counter */

                                delay_count--;
                                }

/* Shift channel model and equalizer shift registers one place to the right */

            signal = next_pn(pn_shift);
                        shift_right_layer(ann.layer_0, ann.nodes_0);
                        shift_right_fir(modelline, m_taps);
                        shift_right_fir(delayline, memory);
                        }

/* Now work out BER by testing over 10000 bits */

                if (test)
                        {
                        gotoxy(x-10, y);
                        printf("TESTING    ");

/* Reset error counter */

                        no_errors = 0;
```

```
/* Reset delay lines */

                        for(i=0; i<10000; i++)
                                {

/* Add next preamble bit into channel model */

                                modelline[0].value = sign((2.0*URAND)-1.0);

/* Calculate signal and add to equalilzer shift register */

                                ann.layer_0[0].output = fir_output(modelline, m_taps) +
noise(snr);


/* Calculate output of equalizer and then work out its error */

                dist(ann.layer_0, ann.layer_1, ann.nodes_0, ann.nodes_1, ann.learn_rate);
                                ffwd_ada(ann.layer_1, ann.layer_2, ann.nodes_1, ann.nodes_2);

                                if (sign(ann.layer_2[0].output) != modelline[0].value)
                                        {
                                        no_errors++;
                                        }

/* Shift channel model and equalizer shift registers one place to the right */

                                shift_right_layer(ann.layer_0, ann.nodes_0);
                                shift_right_fir(modelline, m_taps);
                                }
                        gotoxy(x-25, y);
                        printf("LAST BER:%05d", no_errors);
                        ber += (double)no_errors/10000.0;
                        }
                }

/* Store mean bit error rate in a text file */

        save_doubles(&ber, 1, runs, path, "BER.TXT");

/* Clean up memory */

        rb_tidyup();
        exit(0);
        }
```